Formosa Crypto

End-to-End Formally Verified High-Assurance High-Speed Crypto Software

Santiago Arranz-Olmos **Ignacio Cuevas**

March 9th 2026

Formosa Crypto

End-to-End Formally Verified High-Assurance High-Speed Crypto Software

Santiago Arranz-Olmos **Ignacio Cuevas** Basavesh Ammanaghatta Shivakumar José Bacelar Almeida Gustavo Delerue Manuel Barbosa Francisca Barros Gilles Barthe Lionel Blatter Chitchanok Chuengsatiansup François Dupressoir Luís Esquível Ruben Gonzalez Benjamin Grégoire Andreas Hülsing Vincent Hwang Jan Jancar Vincent Laporte Jean-Christophe Léchenet Ting-han Lim Cameron Low Tiago Oliveira Hugo Pacheco Swarn Priya Miguel Quaresma Rolfe Schmidt Peter Schwabe Antoine Séré Lucas Tabary-Maujean Pierre-Yves Strub Yuval Yarom Zhiyuan Zhang Jieyu Zheng ...

March 9th 2026

Formosa: End-to-End Verified High-Assurance High-Speed Crypto Software

FORMOSA CRYPTO

FORMally verified
Open
Source
Amazing crypto

Boston University
CASA Cyber Security in the Age of Large-Scale Adversaries
institute IMDEA software
Universidade do Minho
Max Planck Institute for Security and Privacy
PQ SHIELD
INESCTEC
SANDBOXAQ
UCLouvain
CryptoExperts
University of Bristol
Inria
h_da hochschule darmstadt
U.PORTO Faculdade de Ciências Universidade do Porto
Rosenpass
The University of Melbourne
TU/e Eindhoven University of Technology

Formosa: End-to-End Verified High-Assurance High-Speed Crypto Software

FORMOSA CRYPTO

FORMally verified
Open
Source
Amazing crypto

BOSTON UNIVERSITY
CASA Cyber Security in the Age of Large-Scale Adversaries
institute IMDEA software
Universidade do Minho
MAX PLANCK INSTITUTE FOR SECURITY AND PRIVACY
PQ SHIELD
INESCTEC
SANDBOXAQ
UCLouvain
CryptoExperts
University of BRISTOL
Inria
h_da hochschule darmstadt
U.PORTO FACULDADE DE CIÊNCIAS UNIVERSIDADE DO PORTO
Rosenpass
THE UNIVERSITY OF MELBOURNE
TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

What Do We Want from Crypto Software?

What Do We Want from Crypto Software?

Scheme

What Do We Want from Crypto Software?

Scheme

Correct

What Do We Want from Crypto Software?

Scheme

Correct

Secure

What Do We Want from Crypto Software?

Scheme	Implementation	
Correct		
Secure		

What Do We Want from Crypto Software?

Scheme	Implementation
Correct	Respects the scheme
Secure	

What Do We Want from Crypto Software?

Scheme	Implementation	
Correct	Respects the scheme	
Secure	Safe	

What Do We Want from Crypto Software?

Scheme	Implementation	Assembly
Correct	Respects the scheme	
Secure	Safe	

What Do We Want from Crypto Software?

Scheme	Implementation	Assembly
Correct	Respects the scheme	Respects the implementation
Secure	Safe	

What Do We Want from Crypto Software?

Scheme	Implementation	Assembly
Correct	Respects the scheme	Respects the implementation
Secure	Safe	Efficient

What Do We Want from Crypto Software?

Scheme	Implementation	Assembly
Correct	Respects the scheme	Respects the implementation
Secure	Safe	Efficient
		Side-Channel secure

What Do We Want from Crypto Software?

Scheme

Implementation

Assembly

Signal Announcement: ML-KEM Integration and Contact Discovery Service

Signal

Signal Announcement: ML-KEM Integration and Contact Discovery Service

Signal

signalapp / **noise-c**

Code | Issues | **Pull requests** 1 | Agents | Actions | Security | Insights

Use latest `mlkem-libjade` #7

Merged **rolfe-signal** merged 16 commits into `main` from `mbbarbosa-changes` last month

Conversation 8 | Commits 16 | Checks 0 | Files changed 12

Signal Announcement: ML-KEM Integration and Contact Discovery Service

Signal

signalapp / noise-c

Code | Issues | Pull requests 1 | Agents | Actions | Security | Insights

Use latest `mlkem-libjade` #7

Merged rolfe-signal merged 16 commits into `main` from `mbbarbosa-changes` last month

Checks 0 | Files changed 12

Commits on Mar 5, 2026

- Jasmin path oram implementation — 3 people authored 2 days ago — Verified — 859f433

Commits on Feb 21, 2026

- Jasmin implementation of sharding — 3 people authored 2 weeks ago — Verified — e8ecd1c
- Jasmin: util library + ratelimit osort — rolfe-signal and tccueves authored 2 weeks ago — Verified — a39e0fd

Commits on Feb 13, 2026

- Update noise-c to use mlkem-libjade for ML-KEM-1024 — rolfe-signal authored 3 weeks ago — Verified — bfd4fa1
- Jasmin Halfsiphash Update. — tccueves authored 3 weeks ago — Verified — 9a887da

Signal Announcement: ML-KEM Integration and Contact Discovery Service

Taiwan, officially the Republic of China (ROC), is a country in East Asia comprising primarily the **island of Taiwan**, formerly known as **Formosa**,

Flag

Image by Henry Szytko

Seal

Image by Jeff Dahl

Signal Announcement: ML-KEM Integration and Contact Discovery Service

Signal

signalapp / noise-c

Code | Issues | Pull requests 1 | Agents | Actions | Security | Insights

Use latest `mlkem-libjade` #7

Merged rolfe-signal merged 16 commits into `main` from `mbbarbosa-changes` last month

Checks 0 | Files changed 12

Commits on Mar 5, 2026

- Jasmin path oram implementation — 3 people authored 2 days ago — Verified — 859f433

Commits on Feb 21, 2026

- Jasmin implementation of sharding — 3 people authored 2 weeks ago — Verified — e8ecd1c
- Jasmin: util library + ratelimit osort — rolfe-signal and tccueves authored 2 weeks ago — Verified — a39e0fd

Commits on Feb 13, 2026

- Update noise-c to use mlkem-libjade for ML-KEM-1024 — rolfe-signal authored 3 weeks ago — Verified — bfd4fa1
- Jasmin Halfsiphash Update. — tccueves authored 3 weeks ago — Verified — 9a887da

Signal Announcement: ML-KEM Integration and Contact Discovery Service

Signal

signalapp / noise-c

Code | Issues | Pull requests 1 | Agents | Actions | Security | Insights

Use latest `mlkem-libjade` #7

Merged rolfe-signal merged 16 commits into `main` from `mbbarbosa-changes` last month

Checks 0 | Files changed 12

Commits on Mar 5, 2026

- Jasmin path oram implementation — 3 people authored 2 days ago — Verified — 859f433

Commits on Feb 21, 2026

- Jasmin implementation of sharding — 3 people authored 2 weeks ago — Verified — e8ecd1c
- Jasmin: util library + ratelimit osort — rolfe-signal and tcoveves authored 2 weeks ago — Verified — a39e6fd

Commits on Feb 13, 2026

- Update noise-c to use mlkem-libjade for ML-KEM-1024 — rolfe-signal authored 3 weeks ago — Verified — bfd4fa1
- Jasmin Halfsiphash Update. — tcoveves authored 3 weeks ago — Verified — 9a887da

Formosa: Achieving End-to-End Security for Crypto Software

EasyCrypt

Jasmin

ASM

Code (manual)

Proof (manual)

Code (auto)

Proof (auto)

Transformation (auto)

Formosa: Achieving End-to-End Security for Crypto Software

EasyCrypt

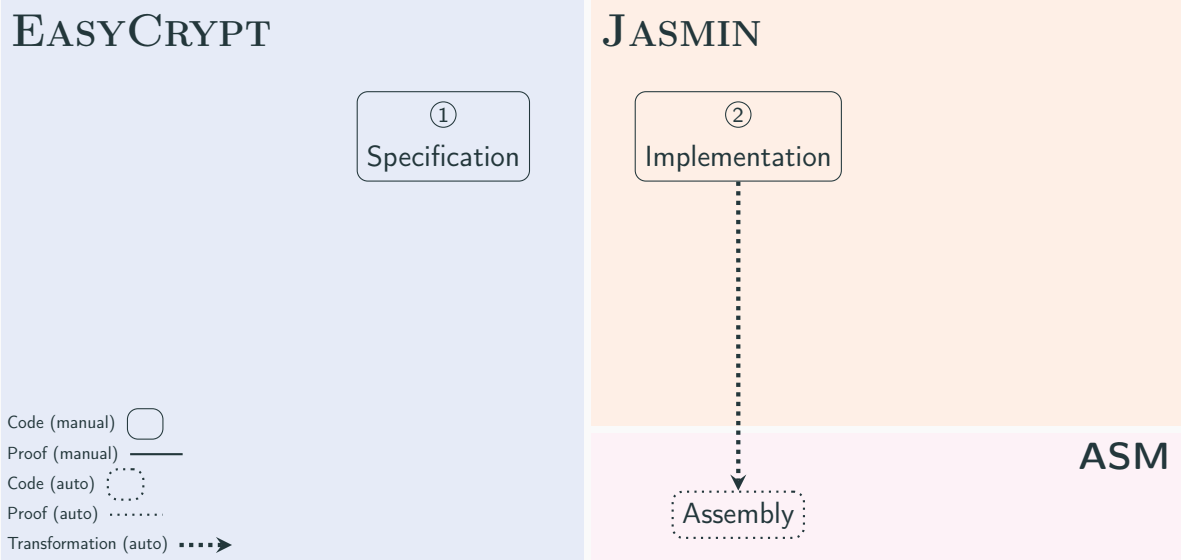
① Specification

Code (manual)
Proof (manual)
Code (auto)
Proof (auto)
Transformation (auto)

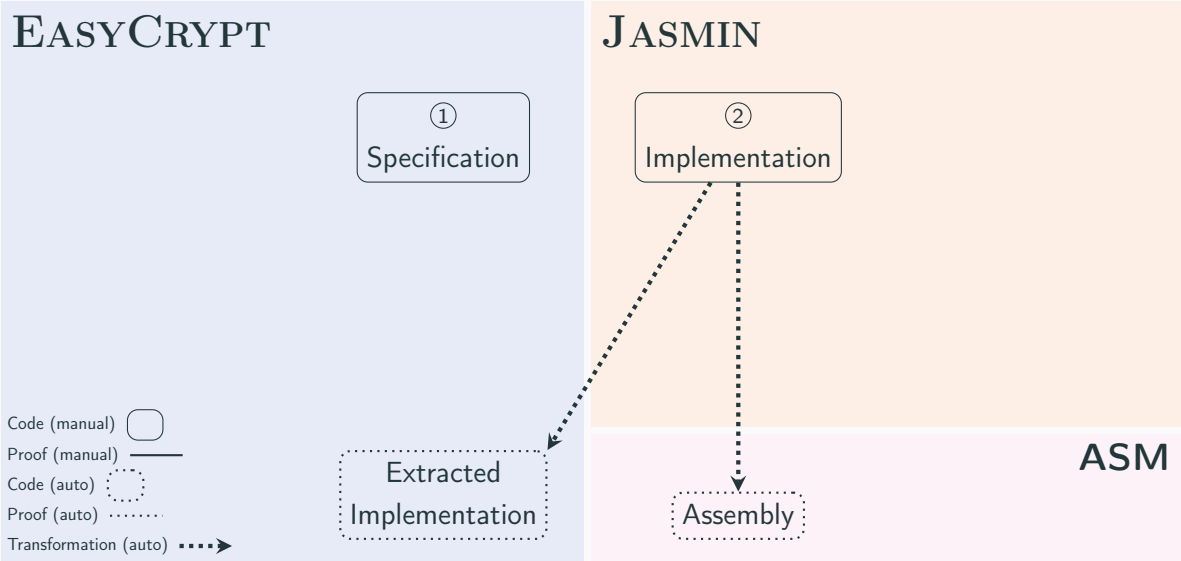
Jasmin

ASM

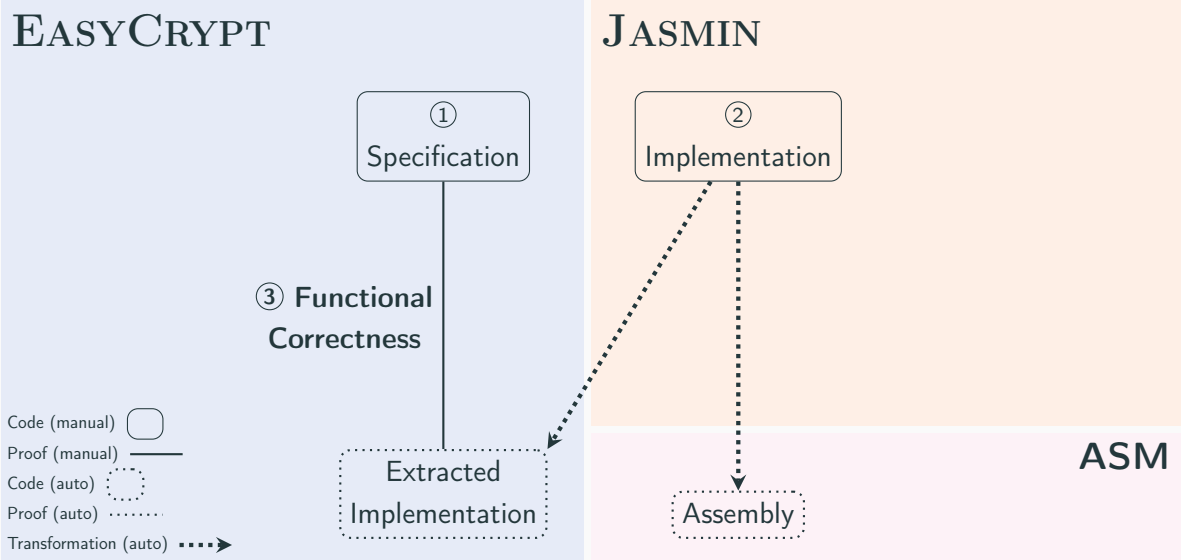
Formosa: Achieving End-to-End Security for Crypto Software



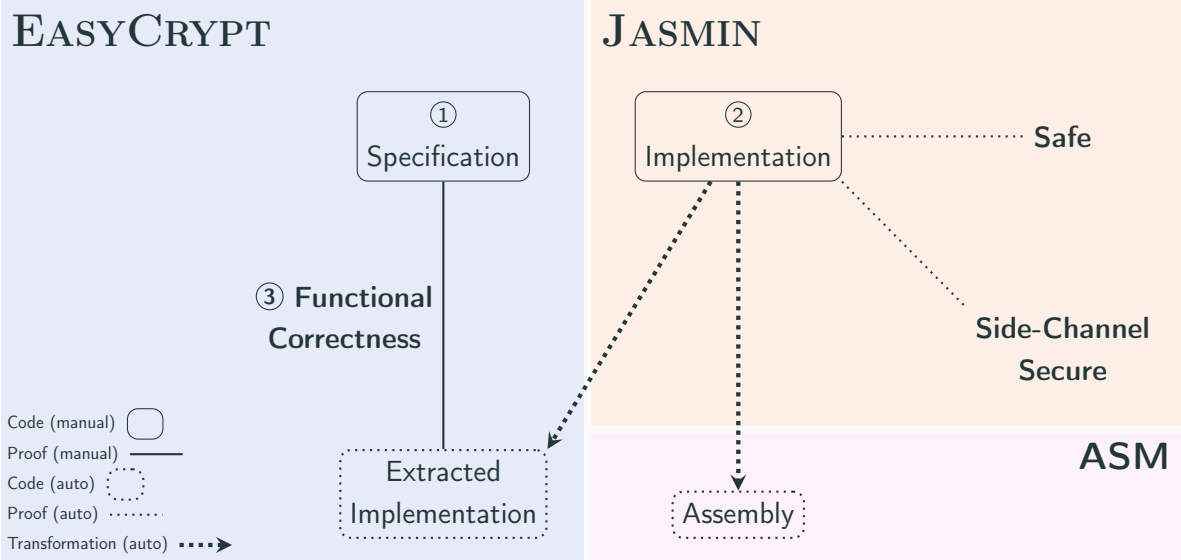
Formosa: Achieving End-to-End Security for Crypto Software



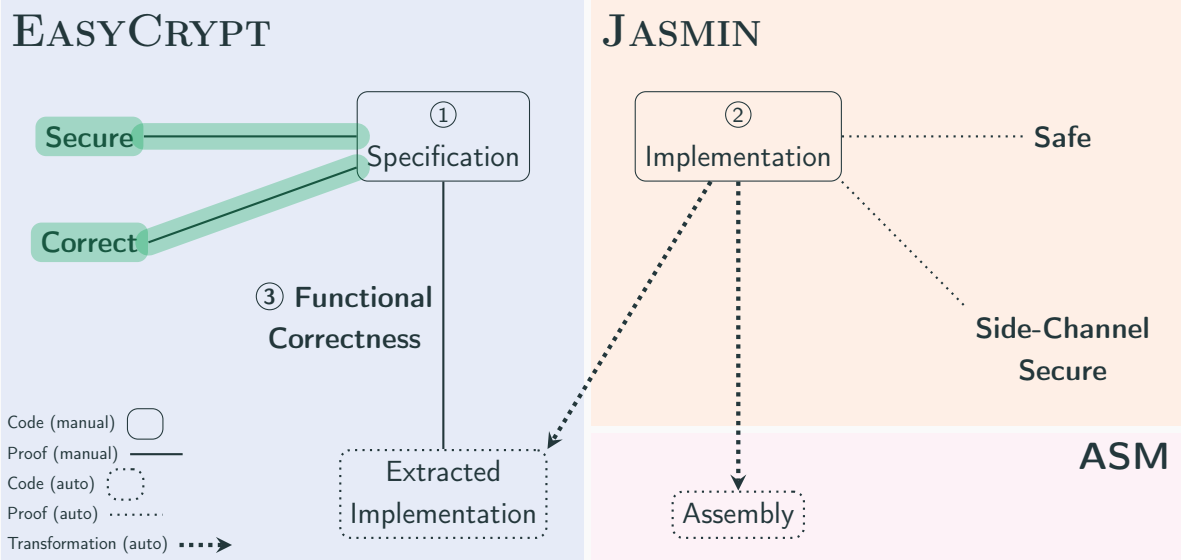
Formosa: Achieving End-to-End Security for Crypto Software



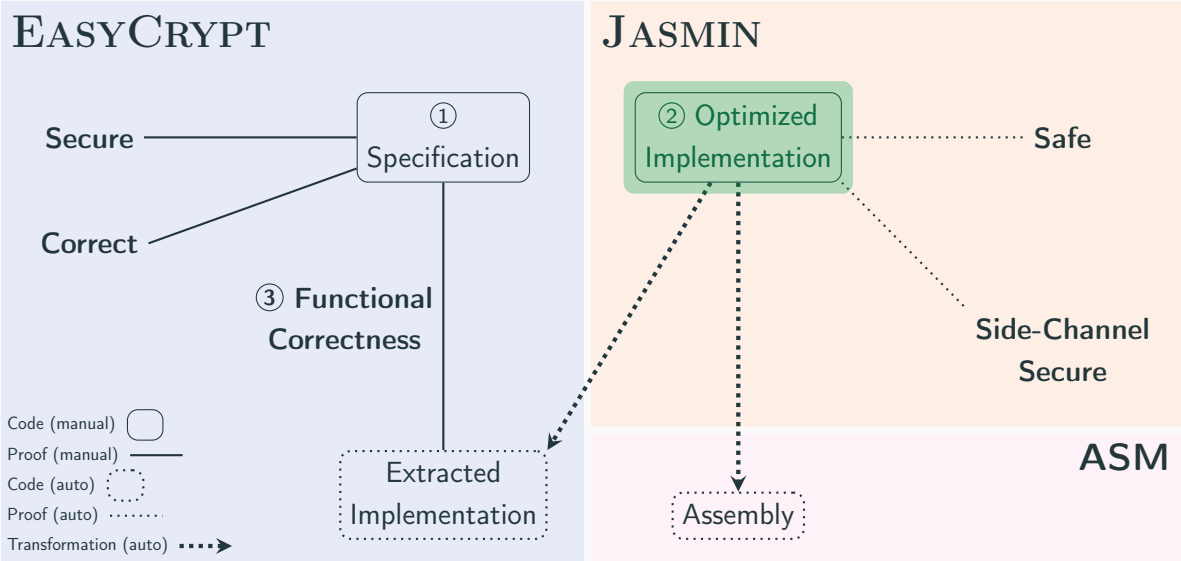
Formosa: Achieving End-to-End Security for Crypto Software



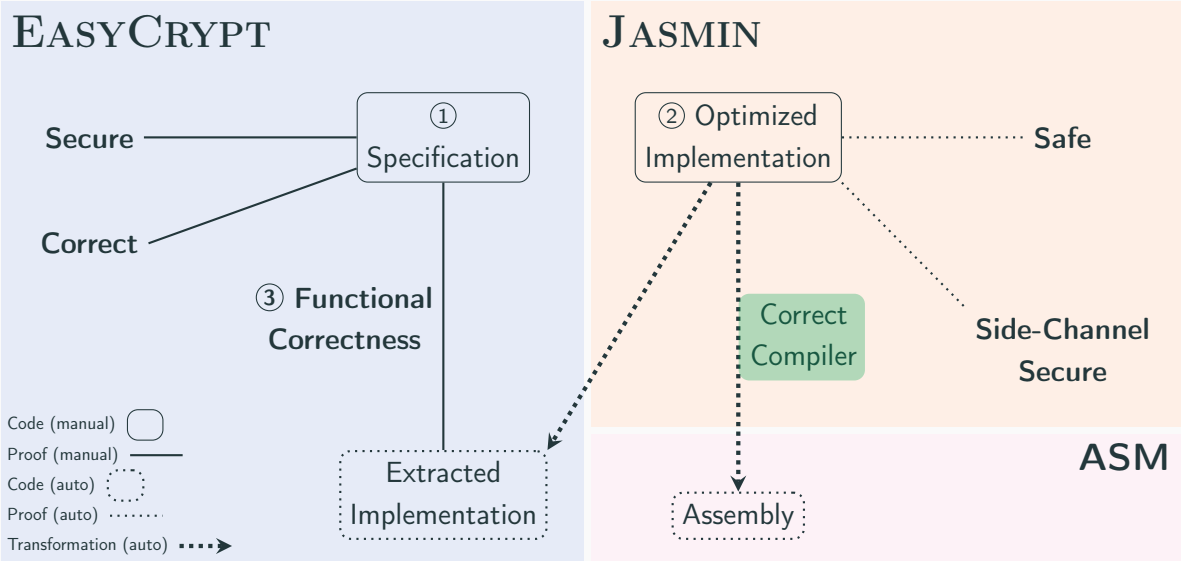
Formosa: Achieving End-to-End Security for Crypto Software



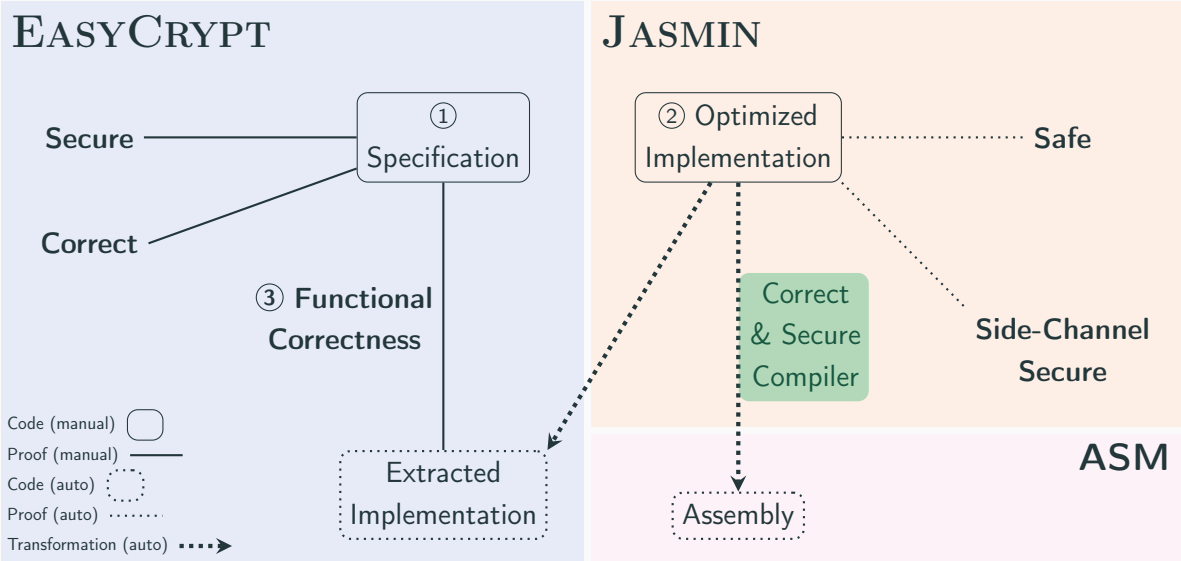
Formosa: Achieving End-to-End Security for Crypto Software



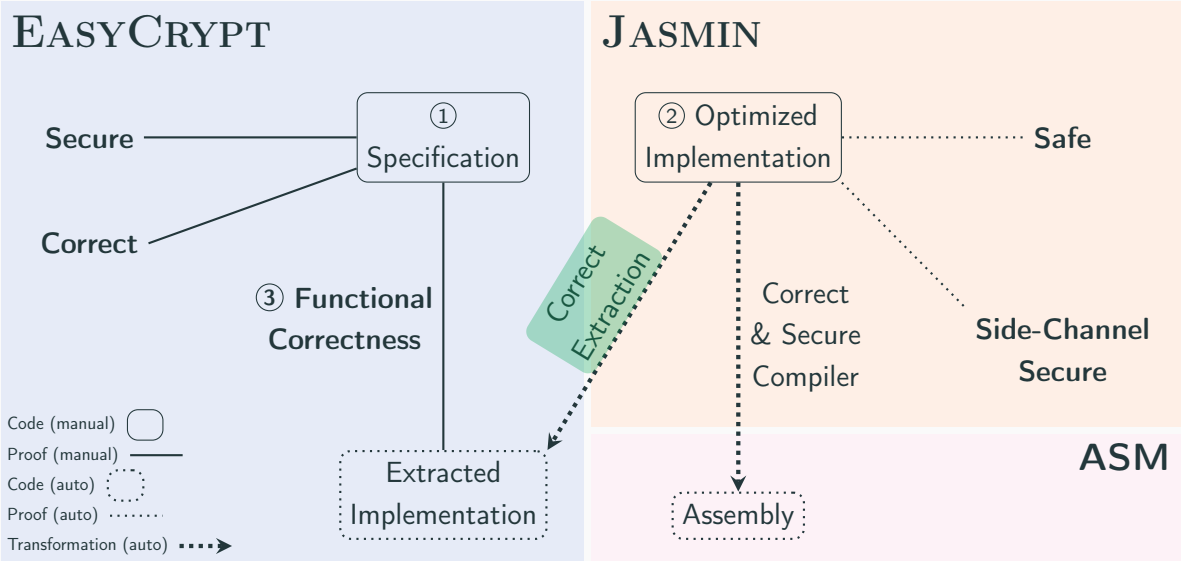
Formosa: Achieving End-to-End Security for Crypto Software



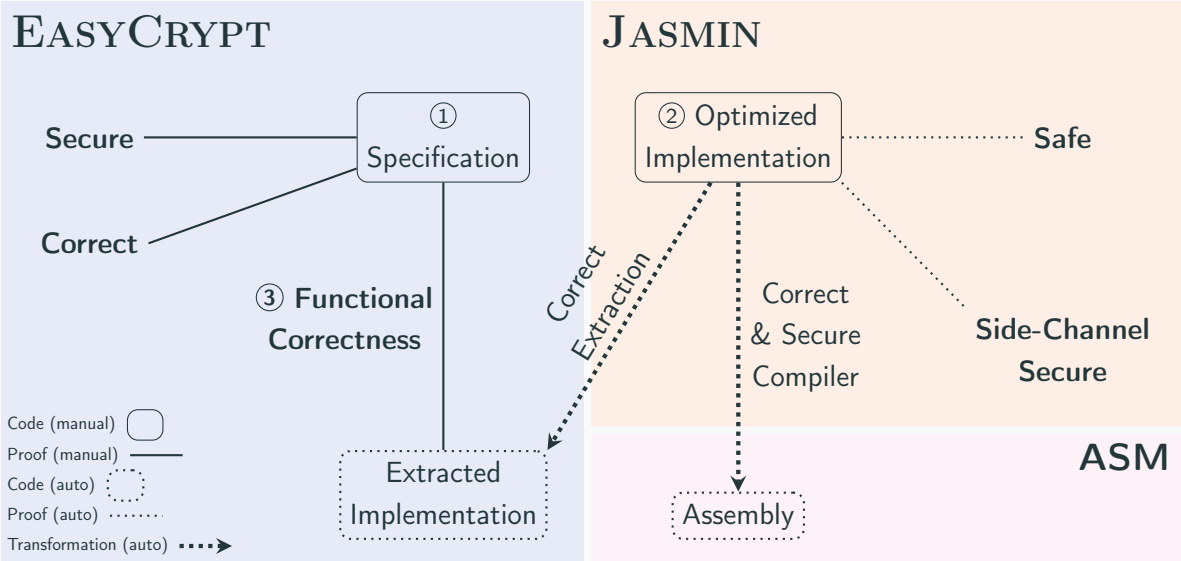
Formosa: Achieving End-to-End Security for Crypto Software



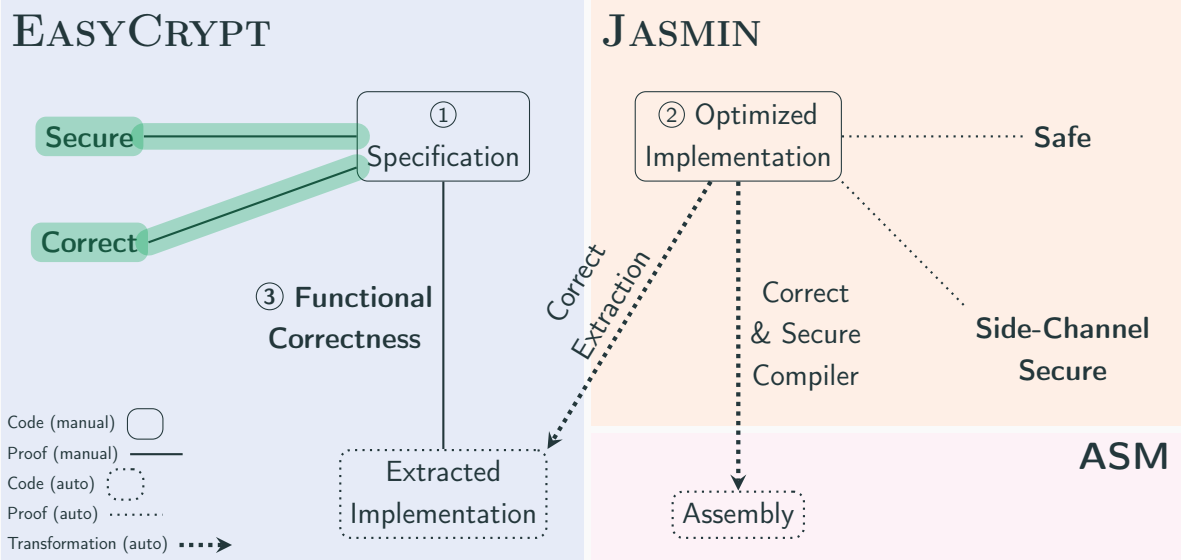
Formosa: Achieving End-to-End Security for Crypto Software



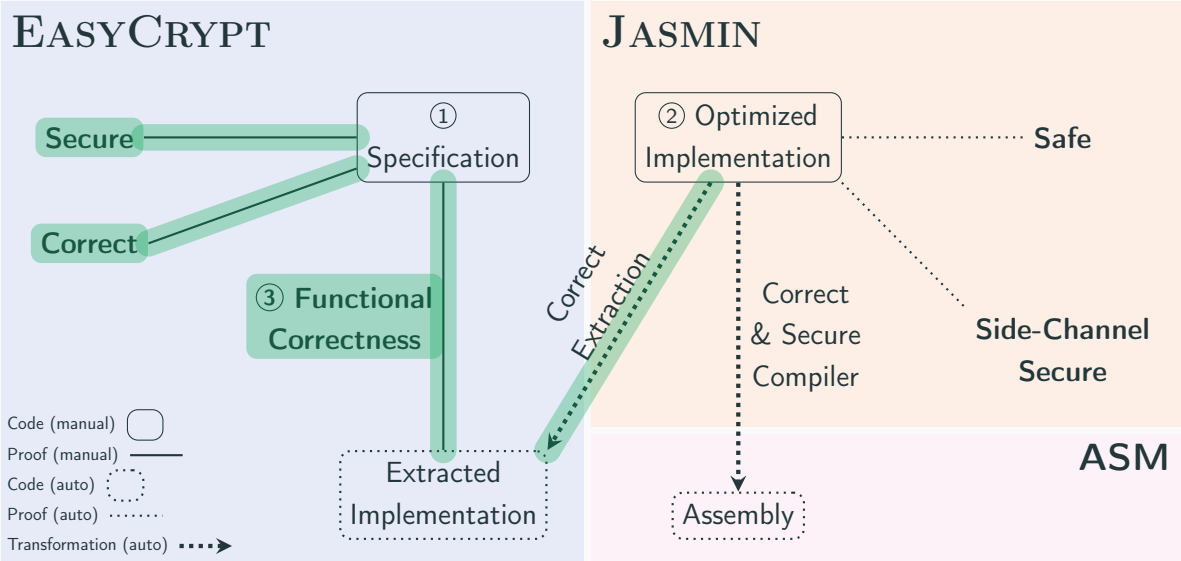
Formosa: Achieving End-to-End Security for Crypto Software



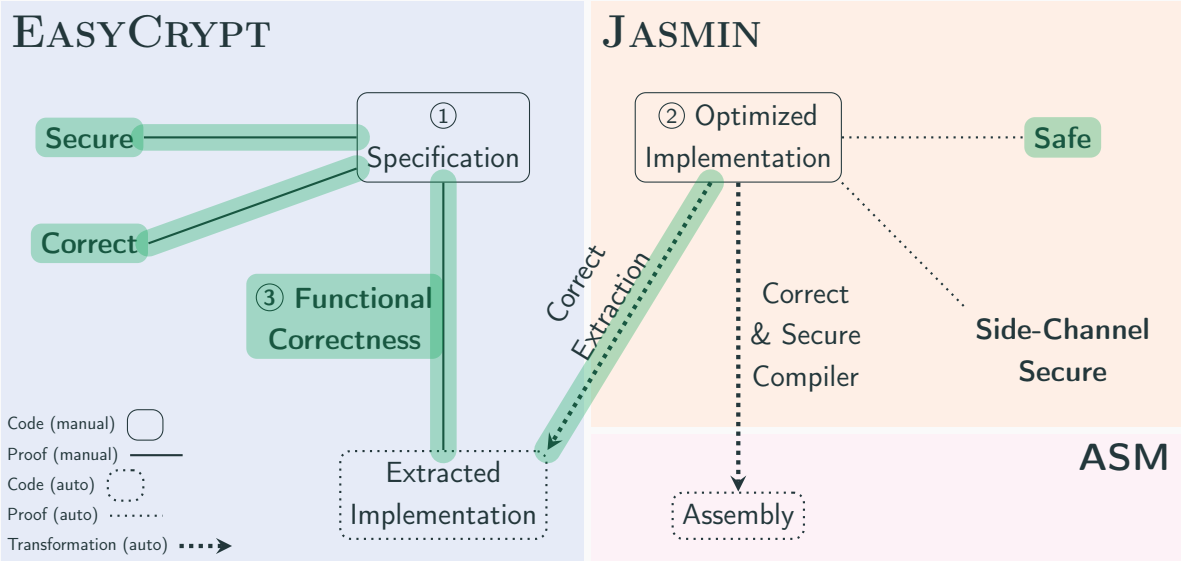
Formosa: Achieving End-to-End Security for Crypto Software



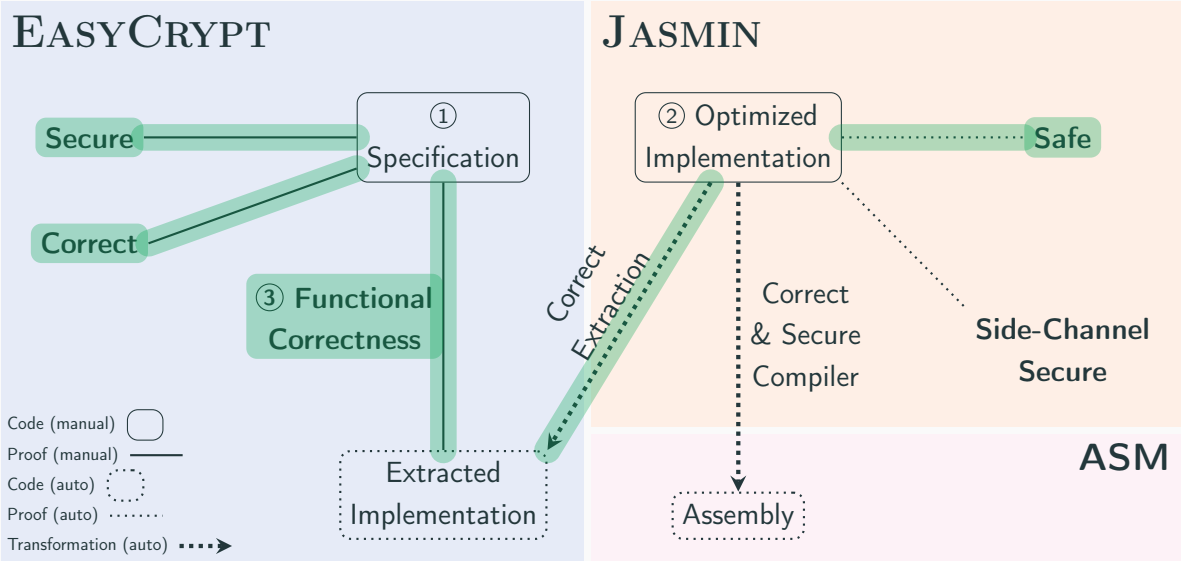
Formosa: Achieving End-to-End Security for Crypto Software



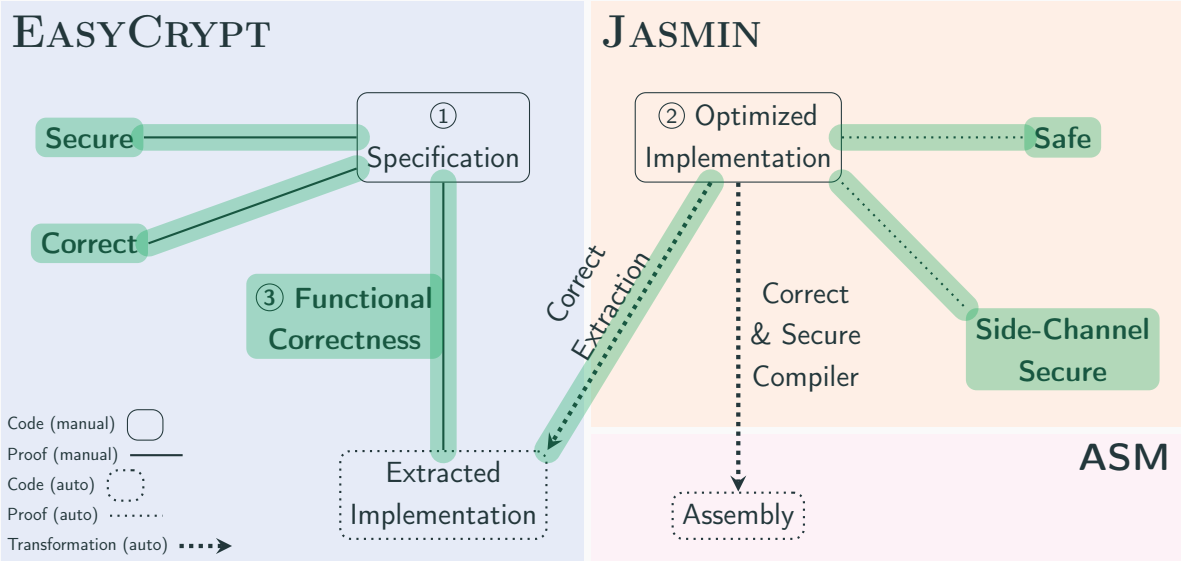
Formosa: Achieving End-to-End Security for Crypto Software



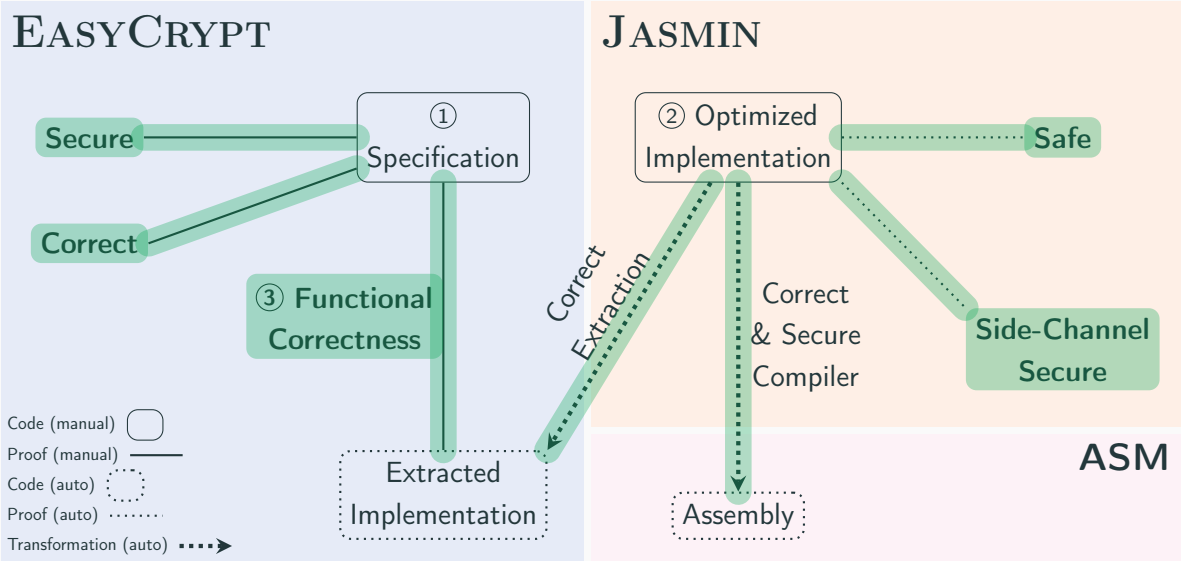
Formosa: Achieving End-to-End Security for Crypto Software



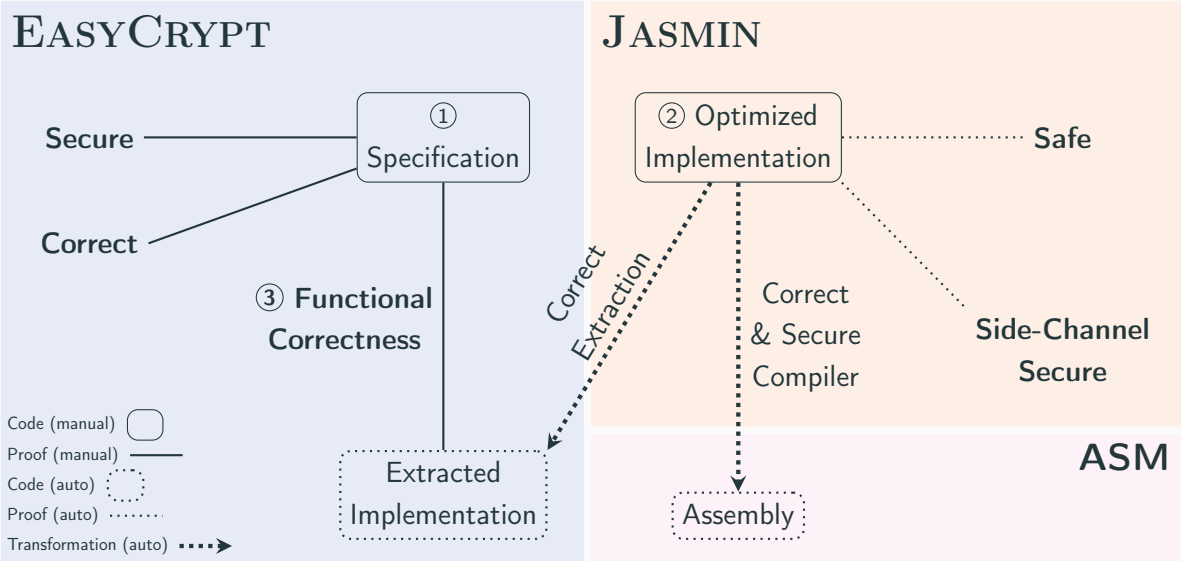
Formosa: Achieving End-to-End Security for Crypto Software



Formosa: Achieving End-to-End Security for Crypto Software



Formosa: Achieving End-to-End Security for Crypto Software

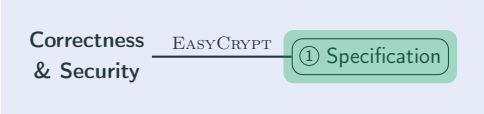


Which ML-KEM implementation should we use?

Ours! Written in JASMIN and verified in EASYCRYPT

Who's JASMIN? What's EASYCRYPT?

Specification: Translating the Standard & Proving Security



Specification: Translating the Standard & Proving Security

Correctness & Security — EasyCrypt — ① Specification

FIPS 203

Federal Information Processing Standards Publication

Module-Lattice-Based Key-Encapsulation Mechanism Standard

Category: Computer Security — Subcategory: Cryptography

3.2 The ML-KEM Scheme

ML-KEM is a key-encapsulation mechanism based on CRYSTALS-Kyber [4], a scheme that was initially described in [8]. The following is a brief and informal description of the computational assumption underlying ML-KEM and how the ML-KEM scheme is constructed.

Specification: Translating the Standard & Proving Security

Correctness & Security — EasyCrypt — ① Specification

FIPS 203

Federal Information Processing Standards Publication

Module-Lattice-Based Key-Encapsulation Mechanism Standard

Category: Computer Security — Subcategory: Cryptography

3.2 The ML-KEM Scheme

ML-KEM is a key-encapsulation mechanism based on CRYSTALS-Kyber [4], a scheme that was initially described in [8]. The following is a brief and informal description of the computational assumption underlying ML-KEM and how the ML-KEM scheme is constructed.

```
module MLKEM = {
  proc kg(coins: W8.t Array.t)
                        : pkey * skey = {
    ...
    s <- nttv s;
    e <- nttv e;
    ...
    pk <- (tv, rho);
    return (pk, sk) ;
  }
  ...
```

Specification: Translating the Standard & Proving Security

Correctness & Security — EasyCrypt — ① Specification

FIPS 203

Federal Information Processing Standards Publication

Module-Lattice-Based Key-Encapsulation Mechanism Standard

Category: Computer Security — Subcategory: Cryptography

3.2 The ML-KEM Scheme

ML-KEM is a key-encapsulation mechanism based on CRYSTALS-Kyber [4], a scheme that was initially described in [8]. The following is a brief and informal description of the computational assumption underlying ML-KEM and how the ML-KEM scheme is constructed.

```
module MLKEM = {
  proc kg(coins: W8.t Array.t)
                        : pkey * skey = {
    ...
    s <- nttv s;
    e <- nttv e;
    ...
    pk <- (tv, rho);
    return (pk, sk);
  }
  ...
```

Specification: Translating the Standard & Proving Security

Correctness & Security — EasyCrypt — ① Specification

FIPS 203

Federal Information Processing Standards Publication

Module-Lattice-Based Key-Encapsulation Mechanism Standard

Category: Computer Security — Subcategory: Cryptography

3.2 The ML-KEM Scheme

ML-KEM is a key-encapsulation mechanism based on CRYSTALS-Kyber [4], a scheme that was initially described in [8]. The following is a brief and informal description of the computational assumption underlying ML-KEM and how the ML-KEM scheme is constructed.

```
module MLKEM = {
  proc kg(coins: W8.t Array.t)
                          : pkey * skey = {
    ...
    s <- nttv s;
    e <- nttv e;
    ...
    pk <- (tv, rho);
    return (pk, sk);
  }
  ...
```

Specification: Translating the Standard & Proving Security

Correctness & Security — EasyCrypt — ① Specification

FIPS 203

Federal Information Processing Standards Publication

Module-Lattice-Based Key-Encapsulation Mechanism Standard

Category: Computer Security — Subcategory: Cryptography

3.2 The ML-KEM Scheme

ML-KEM is a key-encapsulation mechanism based on CRYSTALS-Kyber [4], a scheme that was initially described in [8]. The following is a brief and informal description of the computational assumption underlying ML-KEM and how the ML-KEM scheme is constructed.

```
module MLKEM = {
  proc kg(coins: W8.t Array.t)
                        : pkey * skey = {
    ...
    s <- nttv s;
    e <- nttv e;
    ...
    pk <- (tv, rho);
    return (pk, sk);
  }
  ...
```

Specification: Translating the Standard & Proving Security

Correctness & Security — EasyCrypt — ① Specification

FIPS 203

Federal Information Processing Standards Publication

Module-Lattice-Based Key-Encapsulation Mechanism Standard

Category: Computer Security — Subcategory: Cryptography

3.2 The ML-KEM Scheme

ML-KEM is a key-encapsulation mechanism based on CRYSTALS-Kyber [4], a scheme that was initially described in [8]. The following is a brief and informal description of the computational assumption underlying ML-KEM and how the ML-KEM scheme is constructed.

```
module MLKEM = {
  proc kg(coins: W8.t Array.t)
                          : pkey * skey = {
    ...
    s <- nttv s;
    e <- nttv e;
    ...
    pk <- (tv, rho);
    return (pk, sk) ;
  }
  ...
```

Specification: Translating the Standard & Proving Security

Correctness & Security — EasyCrypt — ① Specification

FIPS 203

Federal Information Processing Standards Publication

Module-Lattice-Based Key-Encapsulation Mechanism Standard

Category: Computer Security — Subcategory: Cryptography

3.2 The ML-KEM Scheme

ML-KEM is a key-encapsulation mechanism based on CRYSTALS-Kyber [4], a scheme that was initially described in [8]. The following is a brief and informal description of the computational assumption underlying ML-KEM and how the ML-KEM scheme is constructed.

```
module MLKEM = {
  proc kg(coins: W8.t Array.t)
                          : pkey * skey = {
    ...
    s <- nttv s;
    e <- nttv e;
    ...
    pk <- (tv, rho);
    return (pk, sk) ;
  }
  ...
```

Specification: Translating the Standard & Proving Security

Correctness & Security — EasyCrypt — ① Specification

FIPS 203

Federal Information Processing Standards Publication

Module-Lattice-Based Key-Encapsulation Mechanism Standard

Category: Computer Security — Subcategory: Cryptography

3.2 The ML-KEM Scheme

ML-KEM is a key-encapsulation mechanism based on CRYSTALS-Kyber [4], a scheme that was initially described in [8]. The following is a brief and informal description of the computational assumption underlying ML-KEM and how the ML-KEM scheme is constructed.

```
module MLKEM = {
  proc kg(coins: W8.t Array.t)
                          : pkey * skey = {
    ...
    s <- nttv s;
    e <- nttv e;
    ...
    pk <- (tv, rho);
    return (pk, sk) ;
  }
  ...
```

```
lemma INDCCA &m : `| Pr[ CCA(MLKEM).main() @&m : res ] - 1/2 | <= (* small *) .
```

Specification: Translating the Standard & Proving Security

Correctness & Security — EasyCrypt — ① Specification

FIPS 203

Federal Information Processing Standards Publication

Module-Lattice-Based Key-Encapsulation Mechanism Standard

Category: Computer Security — Subcategory: Cryptography

3.2 The ML-KEM Scheme

ML-KEM is a key-encapsulation mechanism based on CRYSTALS-Kyber [4], a scheme that was initially described in [8]. The following is a brief and informal description of the computational assumption underlying ML-KEM and how the ML-KEM scheme is constructed.

```
module MLKEM = {
  proc kg(coins: W8.t Array.t)
                          : pkey * skey = {
    ...
    s <- nttv s;
    e <- nttv e;
    ...
    pk <- (tv, rho);
    return (pk, sk) ;
  }
  ...
```

```
lemma INDCCA &m : `| Pr [ CCA(MLKEM).main() @&m : res ] - 1/2 | <= (* small *) .
```

Specification: Translating the Standard & Proving Security

Correctness & Security — EasyCrypt — ① Specification

FIPS 203

Federal Information Processing Standards Publication

Module-Lattice-Based Key-Encapsulation Mechanism Standard

Category: Computer Security — Subcategory: Cryptography

3.2 The ML-KEM Scheme

ML-KEM is a key-encapsulation mechanism based on CRYSTALS-Kyber [4], a scheme that was initially described in [8]. The following is a brief and informal description of the computational assumption underlying ML-KEM and how the ML-KEM scheme is constructed.

```
module MLKEM = {
  proc kg(coins: W8.t Array.t)
                          : pkey * skey = {
    ...
    s <- nttv s;
    e <- nttv e;
    ...
    pk <- (tv, rho);
    return (pk, sk) ;
  }
  ...
```

```
lemma INDCCA &m : `| Pr [ CCA(MLKEM).main() @&m : res ] - 1/2 | <= (* small *) .
```

Implementation: Writing High-Speed Crypto Software

Implementation: Writing High-Speed Crypto Software

② Optimized Implementation

Implementation: Writing High-Speed Crypto Software

```
fn csubq(reg ptr u16[N] rp)
                    -> reg ptr u16[N] {
   reg u64 i  = 0;
  while (i < N) {
    reg u16 t = rp[i];
    t -= Q;
    reg u16 b = t;
    b >>= 15;
    b &= Q;
    t += b;
    rp[i] = t;
    i += 1;
  }
  return rp;
}
```

② Optimized Implementation

Implementation: Writing High-Speed Crypto Software

```
fn csubq(reg  ptr u16[N] rp )
                    -> reg ptr u16[N] {
  reg u64 i = 0;
  while (i < N) {
    reg u16 t = rp[i];
    t -= Q;
    reg u16 b = t;
    b >>= 15;
    b &= Q;
    t += b;
    rp[i] = t;
    i += 1;
  }
  return rp;
}
```

② Optimized Implementation

Implementation: Writing High-Speed Crypto Software

```
fn csubq(reg  ptr u16[N] rp )
                  -> reg ptr u16[N] {
   reg u64 i  = 0;
  while (i < N) {
    reg u16 t = rp[i];
    t -= Q;
    reg u16 b = t;
    b >>= 15;
    b &= Q;
    t += b;
    rp[i] = t;
    i += 1;
  }
  return rp;
}
```

② Optimized Implementation

Assembly-in-the-Head Programming

Implementation: Writing High-Speed Crypto Software

```
fn csubq(reg  ptr u16[N] rp )
                  -> reg ptr u16[N] {
   reg u64 i  = 0;
  while (i < N) {
    reg u16 t = rp[i];
    t -= Q;
    reg u16 b = t;
    b >>= 15;
    b &= Q;
    t += b;
    rp[i] = t;
    i += 1;
  }
  return rp;
}
```

② Optimized Implementation

Assembly-in-the-Head Programming

Security

Implementation: Writing High-Speed Crypto Software

```
fn csubq(reg  ptr u16[N] rp )
                    -> reg ptr u16[N] {
    reg u64 i  = 0;
   while (i < N) {
      reg u16 t = rp[i];
      t -= Q;
      reg u16 b = t;
      b >>= 15;
      b &= Q;
      t += b;
      rp[i] = t;
      i += 1;
   }
   return rp;
}
```

② Optimized Implementation

Assembly-in-the-Head Programming

Security & Performance

Implementation: Writing High-Speed Crypto Software

```
fn csubq(reg  ptr u16[N] rp )
                    -> reg ptr u16[N] {
    reg u64 i  = 0;
   while (i < N) {
      reg u16 t = rp[i];
      t -= Q;
      reg u16 b = t;
      b >>= 15;
      b &= Q;
      t += b;
      rp[i] = t;
      i += 1;
   }
   return rp;
}
```

② Optimized Implementation

```
inline fn deinterleave( reg u256 z a0 a1 )
                        -> reg u256, reg u256 {
   reg u256 al =  #VPBLEND_16u16 (a0, z, 0xAA);
   reg u256 ah = #VPBLEND_16u16(a1, z, 0xAA);
   al = #VPACKUS_8u32(al, ah);
   a0 = #VPSRL_8u32(a0, 16);
   a1 = #VPSRL_8u32(a1, 16);
   ah = #VPACKUS_8u32(a0, a1);
   return al, ah;
}
```

Implementation: Writing High-Speed Crypto Software

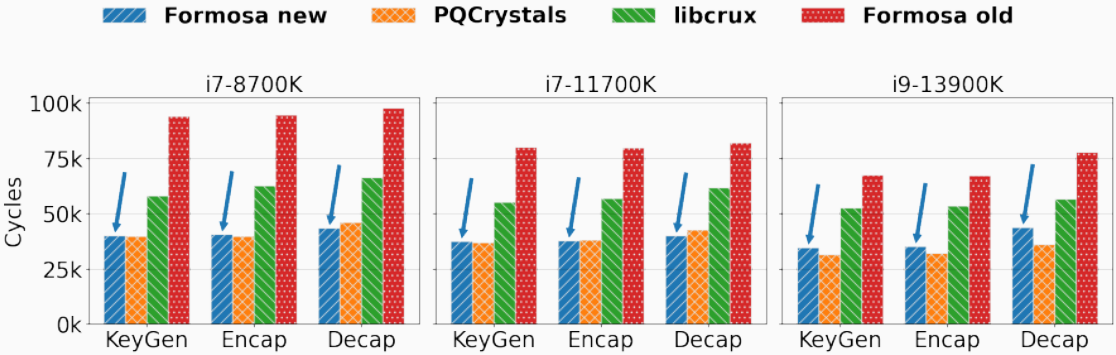
```
fn csubq(reg  ptr u16[N] rp )
                  -> reg ptr u16[N] {
   reg u64 i  = 0;
   while (i < N) {
     reg u16 t = rp[i];
     t -= Q;
     reg u16 b = t;
     b >>= 15;
     b &= Q;
     t += b;
     rp[i] = t;
     i += 1;
   }
   return rp;
}
```

② Optimized Implementation

```
inline fn deinterleave( reg u256 z a0 a1 )
                      -> reg u256, reg u256 {
   reg u256 al =  #VPBLEND_16u16 (a0, z, 0xAA);
   reg u256 ah = #VPBLEND_16u16(a1, z, 0xAA);
   al = #VPACKUS_8u32(al, ah);
   a0 = #VPSRL_8u32(a0, 16);
   a1 = #VPSRL_8u32(a1, 16);
   ah = #VPACKUS_8u32(a0, a1);
   return al, ah;
}
```

Implementation: Closer to State-of-the-Art Performance

Implementation: Closer to State-of-the-Art Performance



Implementation: Contact Discovery Service

Implementation: Contact Discovery Service

path_oram

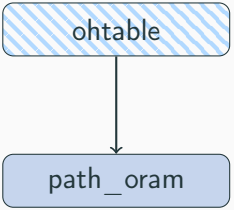
Fully ported

Partially ported

Module dependency

Submodule dependency (in noise-c)

Implementation: Contact Discovery Service



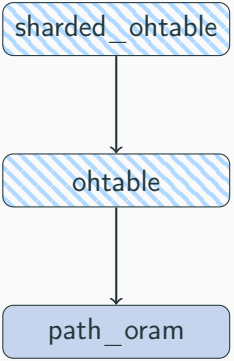
Fully ported

Partially ported

Module dependency

Submodule dependency (in noise-c)

Implementation: Contact Discovery Service



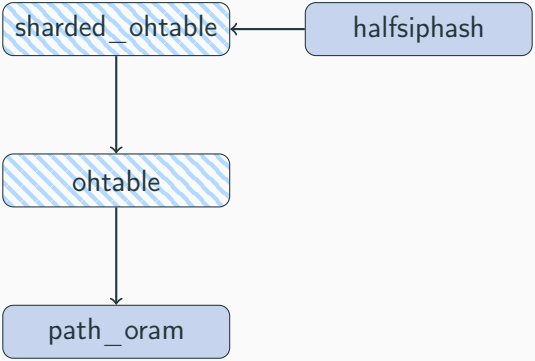
Fully ported

Partially ported

Module dependency

Submodule dependency (in noise-c)

Implementation: Contact Discovery Service



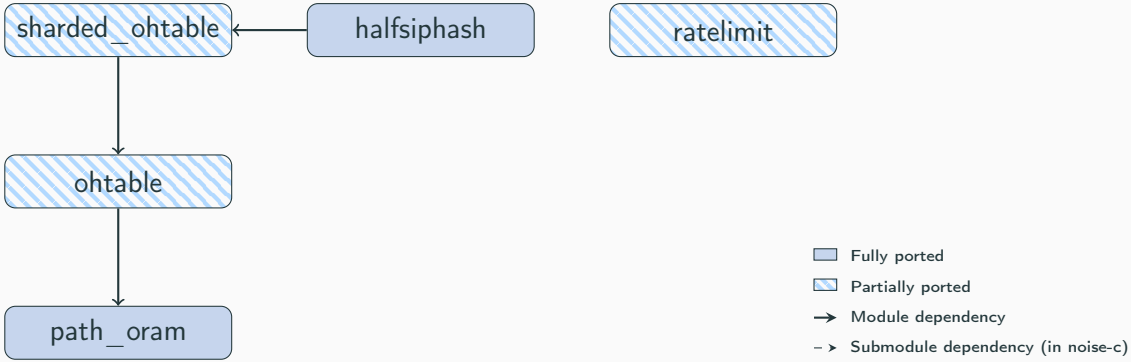
Fully ported

Partially ported

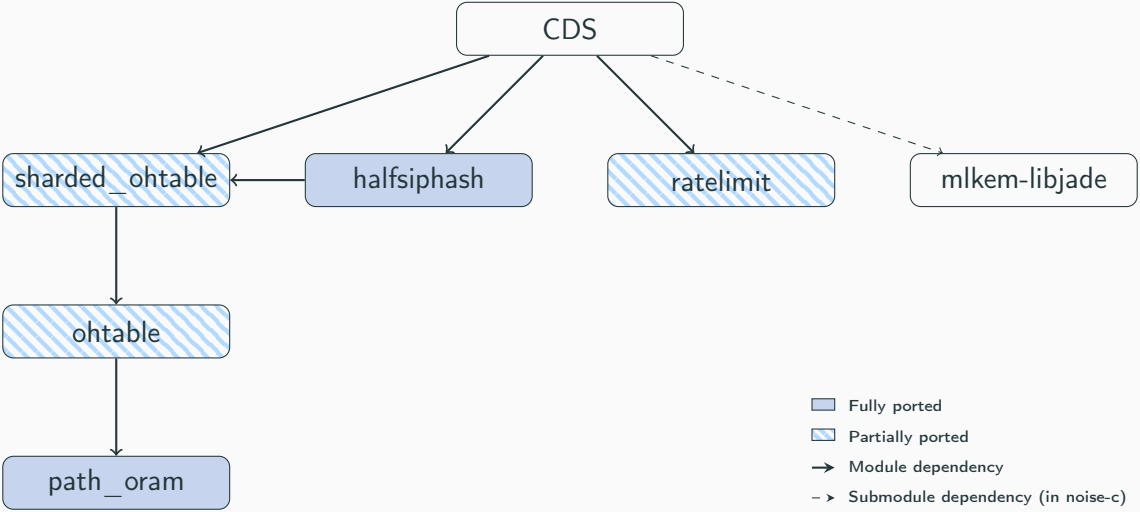
Module dependency

Submodule dependency (in noise-c)

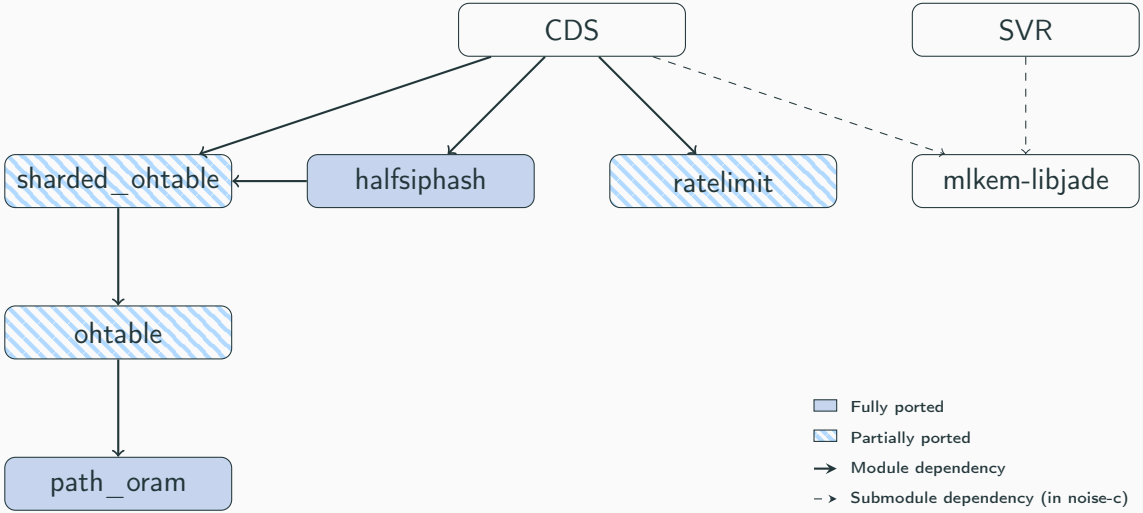
Implementation: Contact Discovery Service



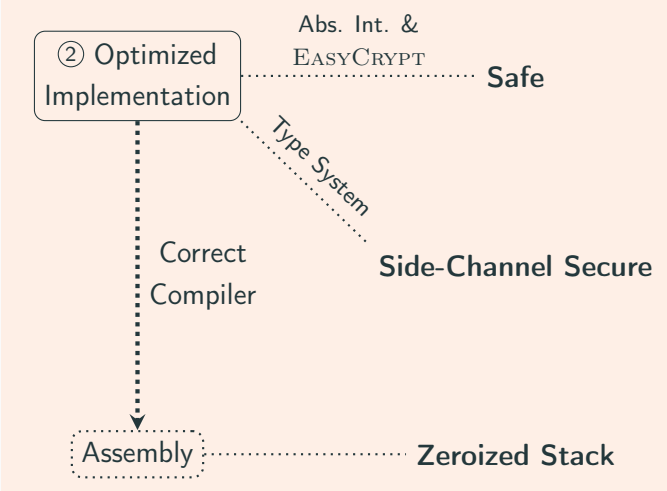
Implementation: Contact Discovery Service



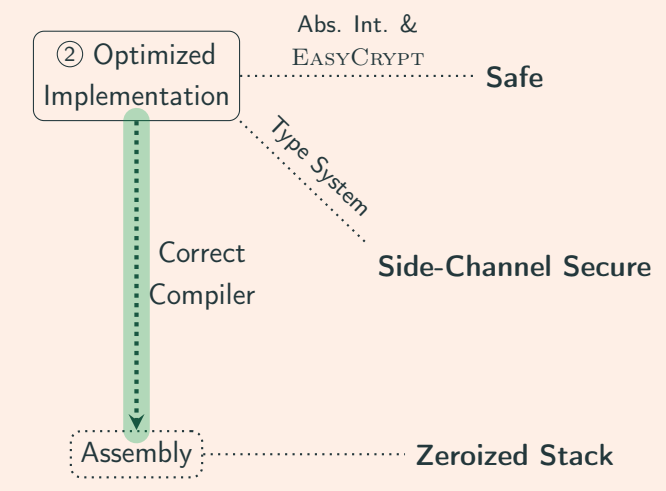
Implementation: Contact Discovery Service



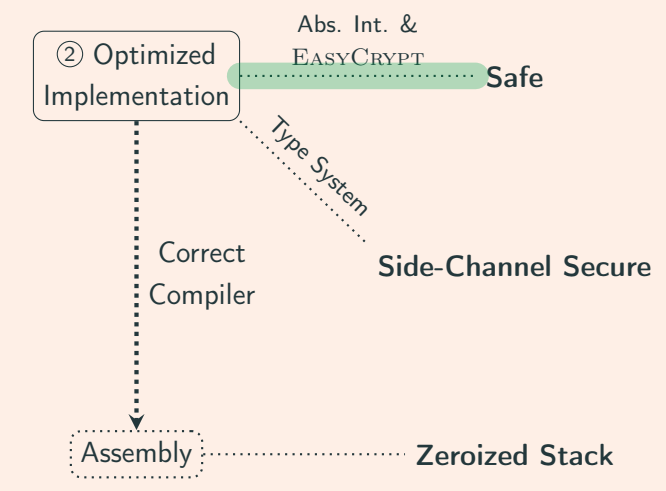
Implementation: Guarantees



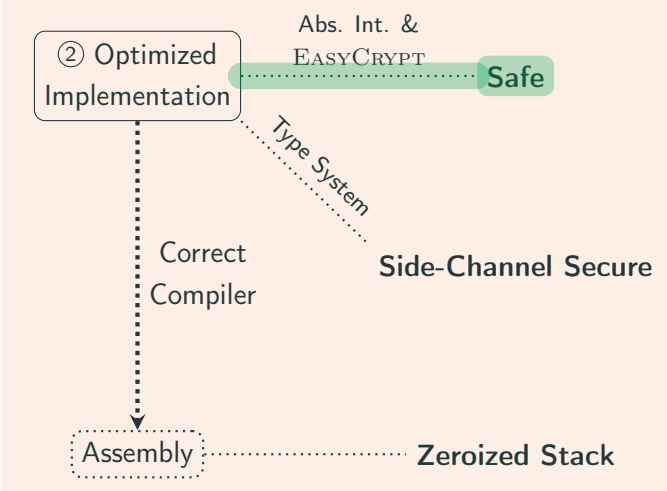
Implementation: Guarantees



Implementation: Guarantees



Implementation: Guarantees



In-bounds mem access

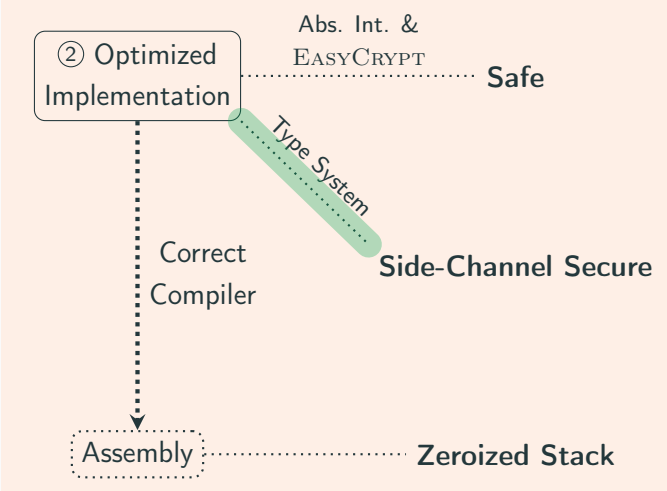
Type-safe

Division by zero

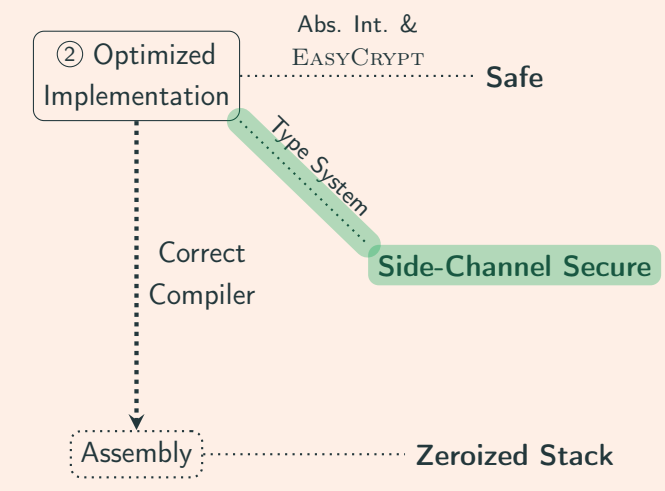
Uninitialized read

...

Implementation: Guarantees



Implementation: Guarantees



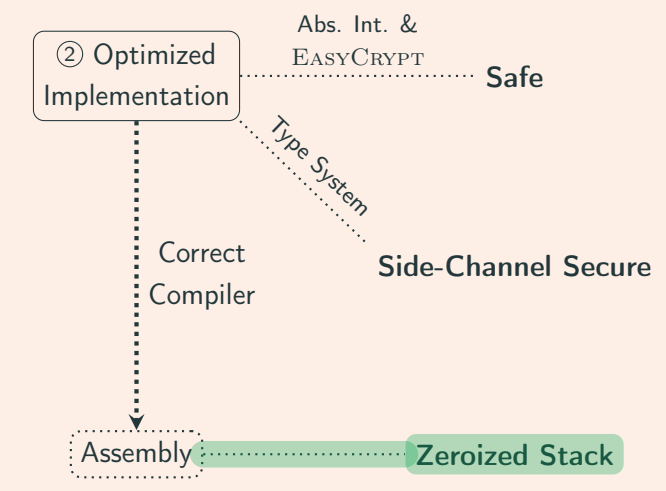
Constant-time

DOIT

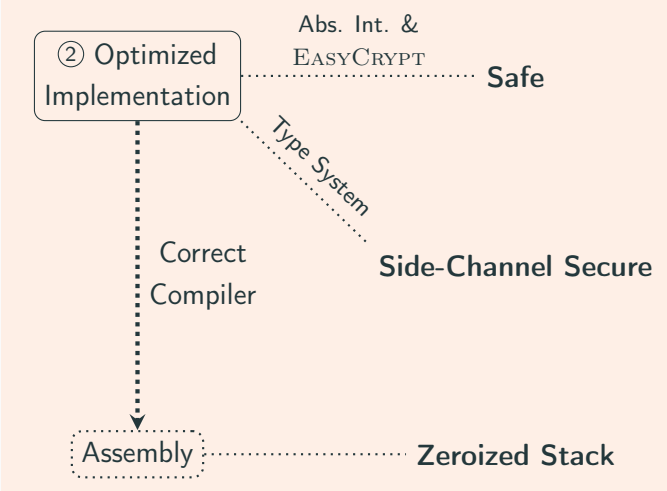
Spectre-v1

Spectre-RSB

Implementation: Guarantees



Implementation: Guarantees



Non-constant-time Logic in CDS Source Code

Close leak in position map

rolfe-signal authored on Mar 14, 2025 Verified fc7ab46

```
85        size_t entries_per_block = oram_block_size(oram_position_map->oram);
86   -    return oram_position_map->base_block_id + (index / entries_per_block);
```

```
98        size_t entries_per_block = oram_block_size(oram_position_map->oram);
99   +    size_t offset = ct_div(index,
100  +        entries_per_block,
101  +        oram_position_map->entries_per_block_ct_m_prime,
102  +        oram_position_map->entries_per_block_ct_shift1,
103  +        oram_position_map->entries_per_block_ct_shift2);
104  +    return oram_position_map->base_block_id + offset;
```

Non-constant-time Logic in CDS Source Code

Close leak in position map

rolfe-signal authored on Mar 14, 2025 Verified fc7ab46

```
85        size_t entries_per_block = oram_block_size(oram_position_map->oram);
86   -    return oram_position_map->base_block_id + (index / entries_per_block);
```

```
98        size_t entries_per_block = oram_block_size(oram_position_map->oram);
99   +    size_t offset = ct_div(index,
100  +        entries_per_block,
101  +        oram_position_map->entries_per_block_ct_m_prime,
102  +        oram_position_map->entries_per_block_ct_shift1,
103  +        oram_position_map->entries_per_block_ct_shift2);
104  +    return oram_position_map->base_block_id + offset;
```

Compiler-Introduced Side-Channel Leakage in CDS

Compiler-Introduced Side-Channel Leakage in CDS

```
void bitonic_merge(...)  {
  ...
  for(...)  {
    bool cond = comp_blocks(...);

     cond_swap_blocks (cond, ...);
     cond_obv_swap_u64 (cond, ...);

  } ... }
```

Compiler-Introduced Side-Channel Leakage in CDS

```
void bitonic_merge(...) {
  ...
  for(...) {
    bool cond = comp_blocks (...);

    cond_swap_blocks (cond, ...);
    cond_obv_swap_u64 (cond, ...);

  } ... }
```

Compiler-Introduced Side-Channel Leakage in CDS

```
void bitonic_merge(...)  {
  ...
  for(...)  {
    bool cond =  comp_blocks (...);

     cond_swap_blocks (cond, ...);
     cond_obv_swap_u64 (cond, ...);

  } ... }
```

$\Longrightarrow$

```
void bitonic_merge(...)  {
  ...
  for(...)  {
    bool cond = comp_blocks(...);
     if (cond) {
       cond_swap_blocks(cond, ...);
       cond_obv_swap_u64(cond, ...);
    }
  } ... }
```

Compiler-Introduced Side-Channel Leakage in CDS

```
void bitonic_merge(...)  {
  ...
  for(...)  {
    bool cond =  comp_blocks (...);

     cond_swap_blocks (cond, ...);
     cond_obv_swap_u64 (cond, ...);

  } ... }
```

$\Longrightarrow$

```
void bitonic_merge(...)  {
  ...
  for(...)  {
    bool cond = comp_blocks(...);
    if (cond) {
      cond_swap_blocks(cond, ...);
      cond_obv_swap_u64(cond, ...);
    }
  } ... }
```

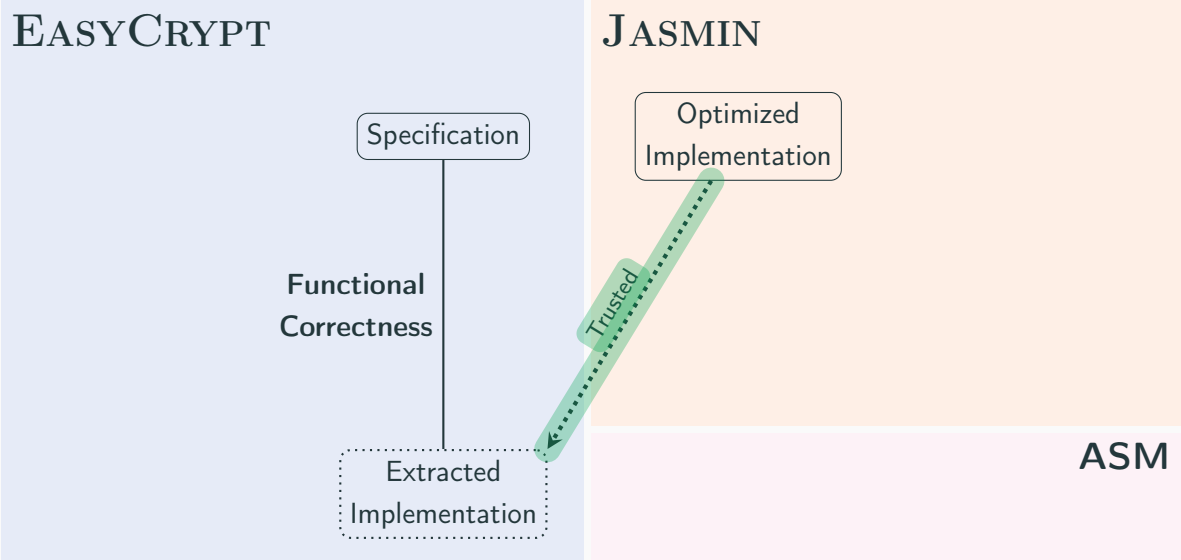
Functional Correctness: Closing the Loop

EasyCrypt

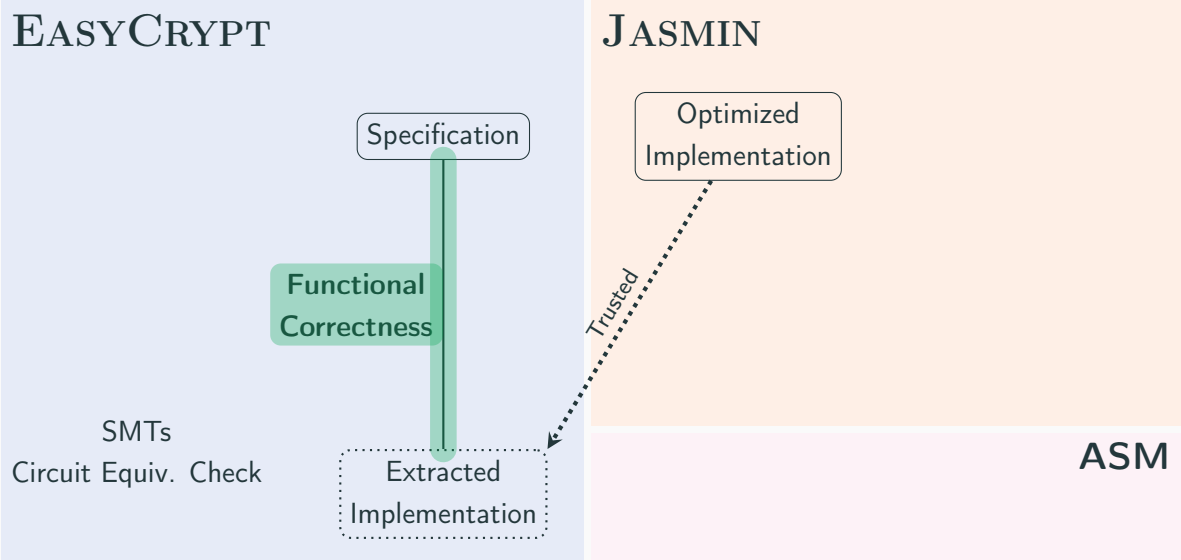
Jasmin

ASM

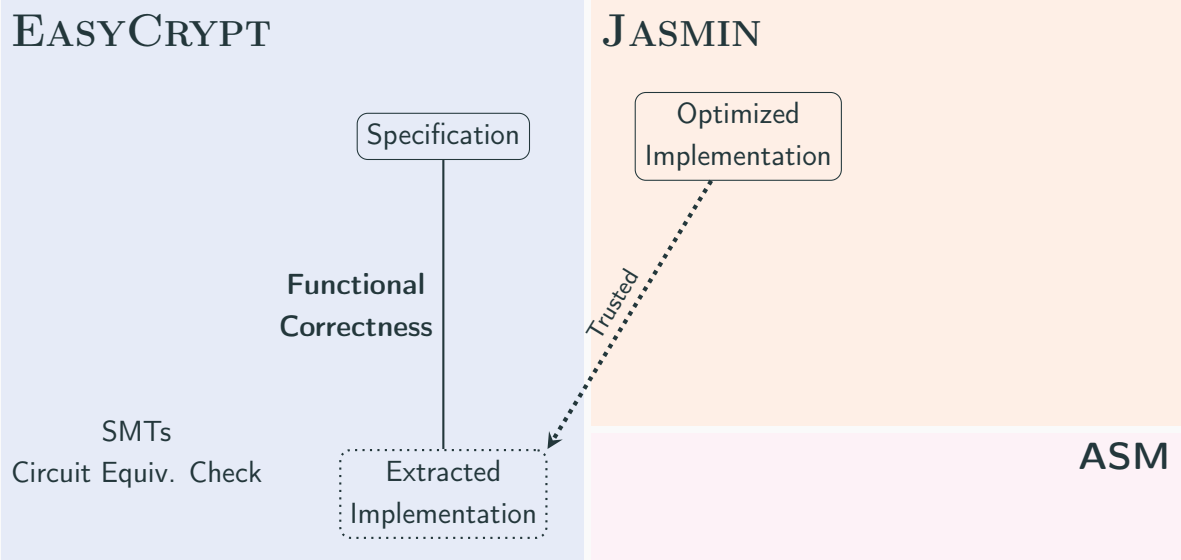
Functional Correctness: Closing the Loop



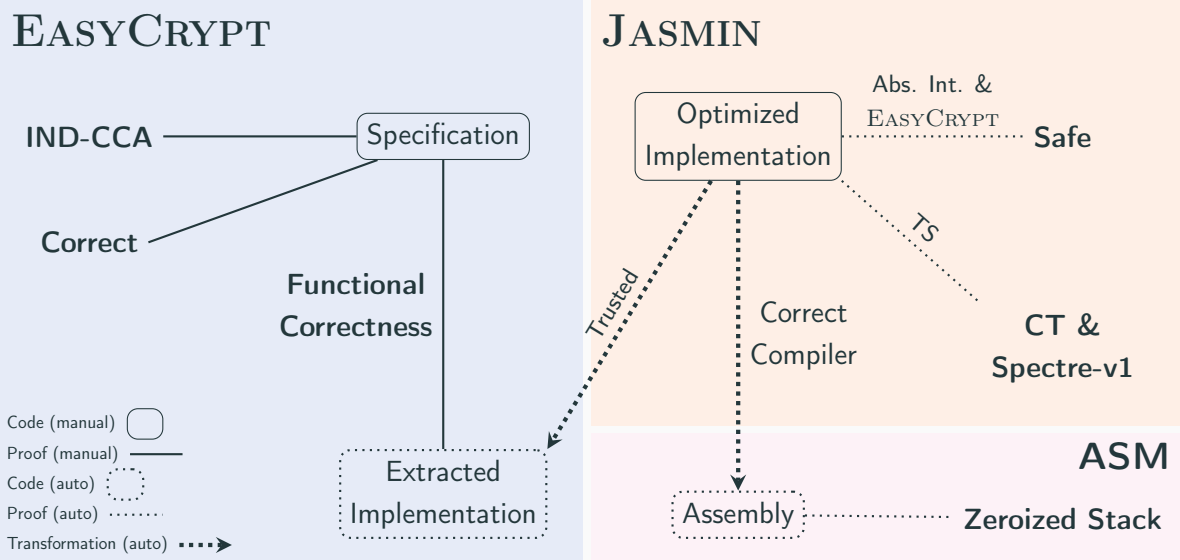
Functional Correctness: Closing the Loop



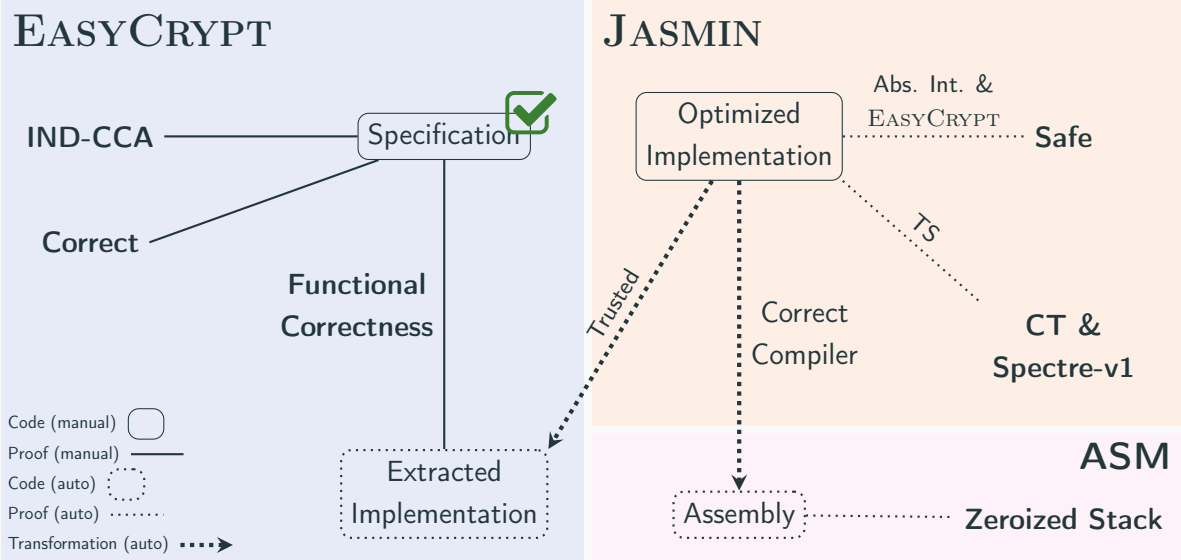
Functional Correctness: Closing the Loop



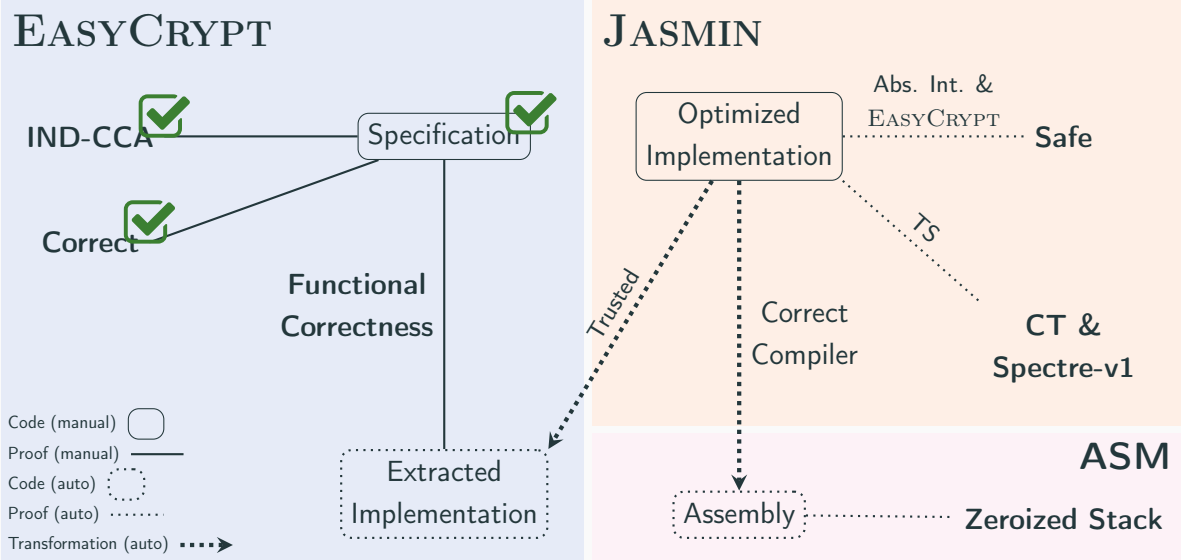
Today's End-to-End Results for ML-KEM



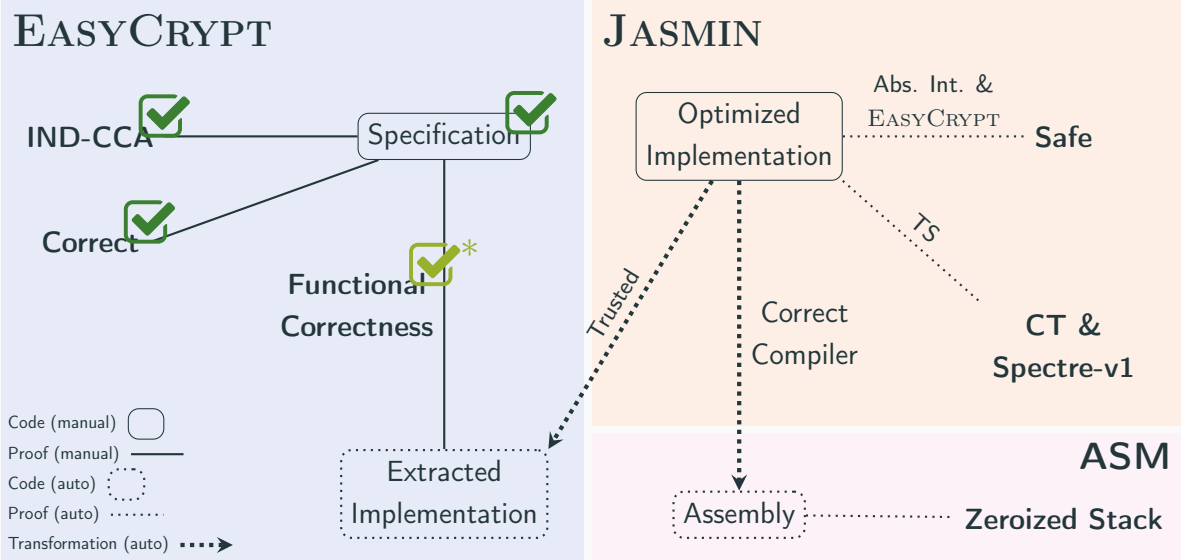
Today's End-to-End Results for ML-KEM



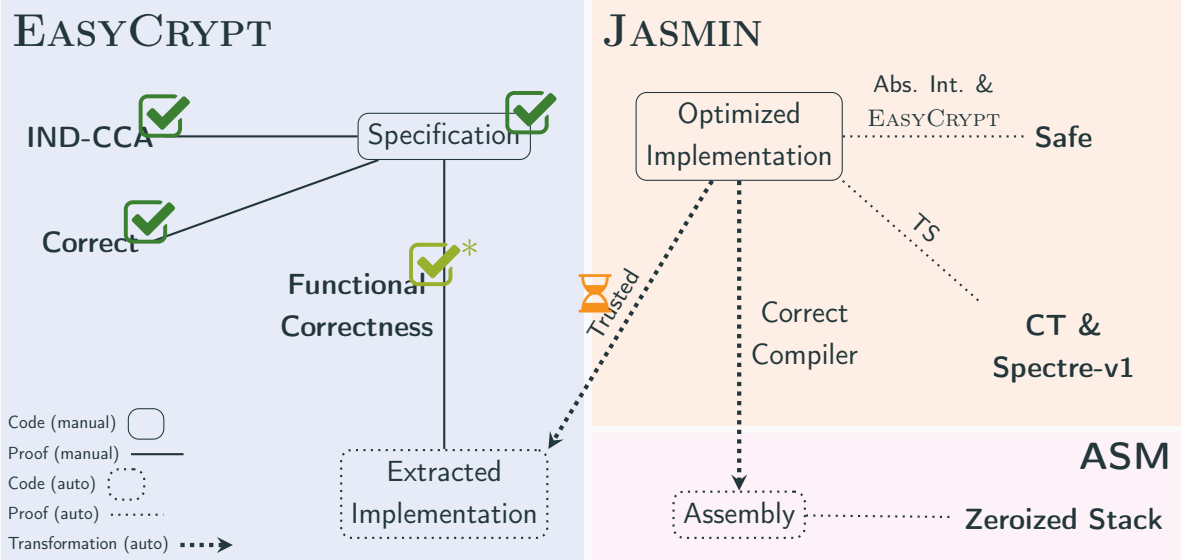
Today's End-to-End Results for ML-KEM



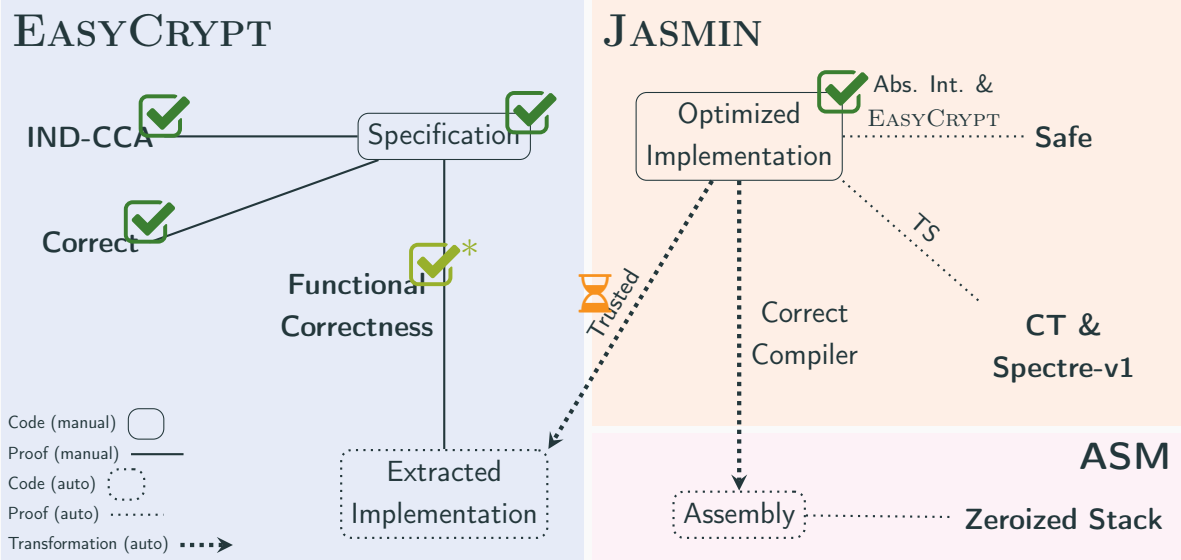
Today's End-to-End Results for ML-KEM



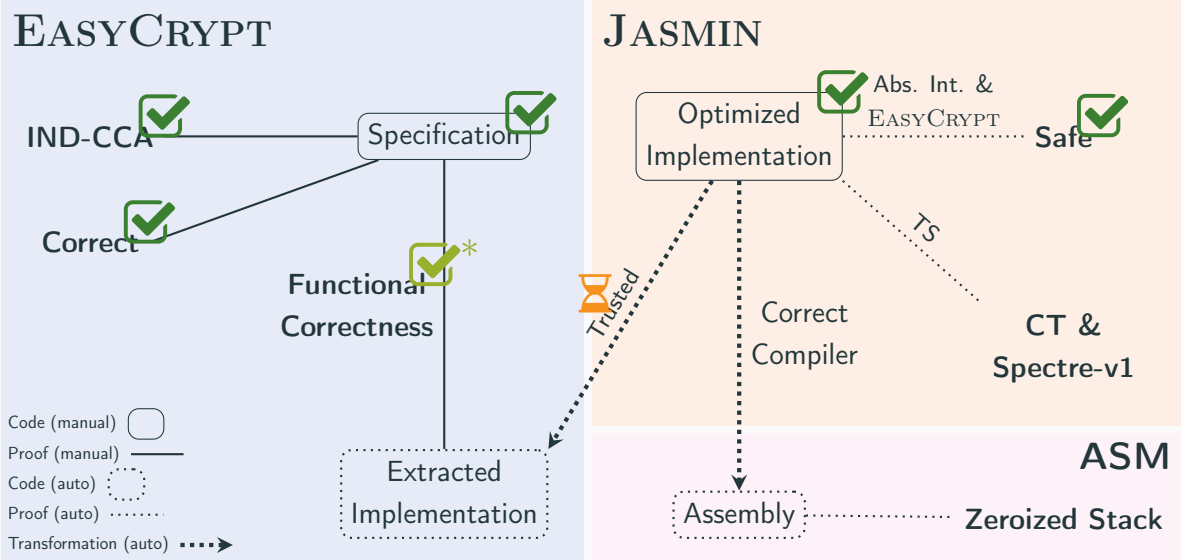
Today's End-to-End Results for ML-KEM



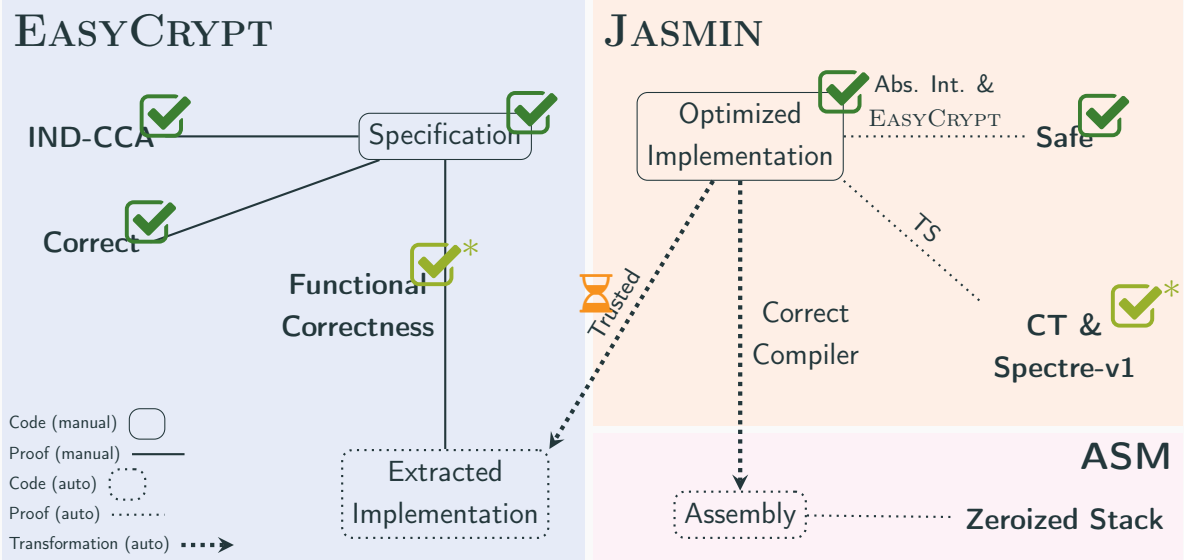
Today's End-to-End Results for ML-KEM



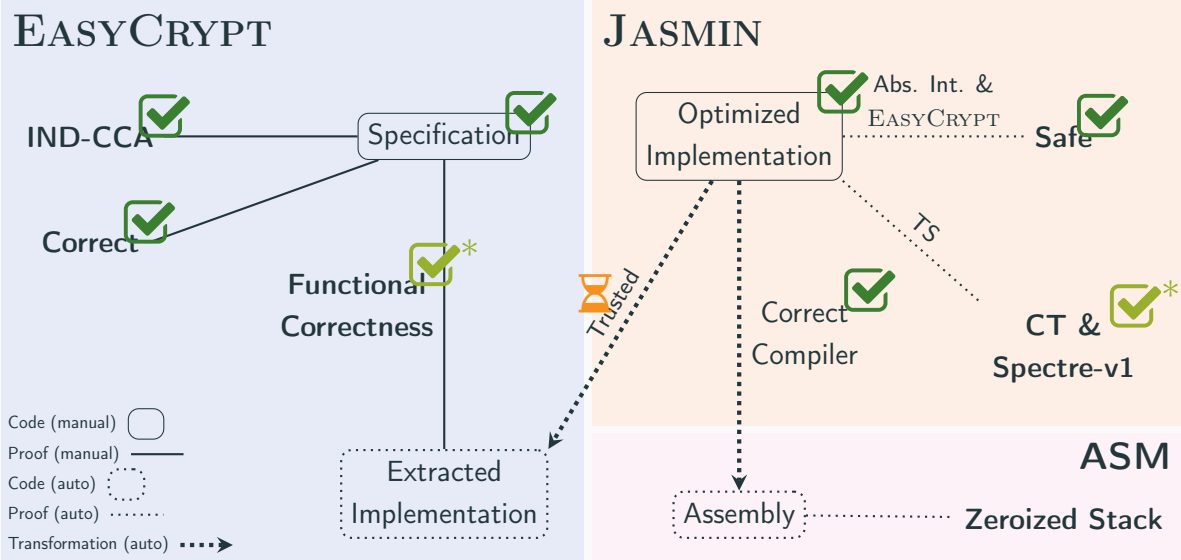
Today's End-to-End Results for ML-KEM



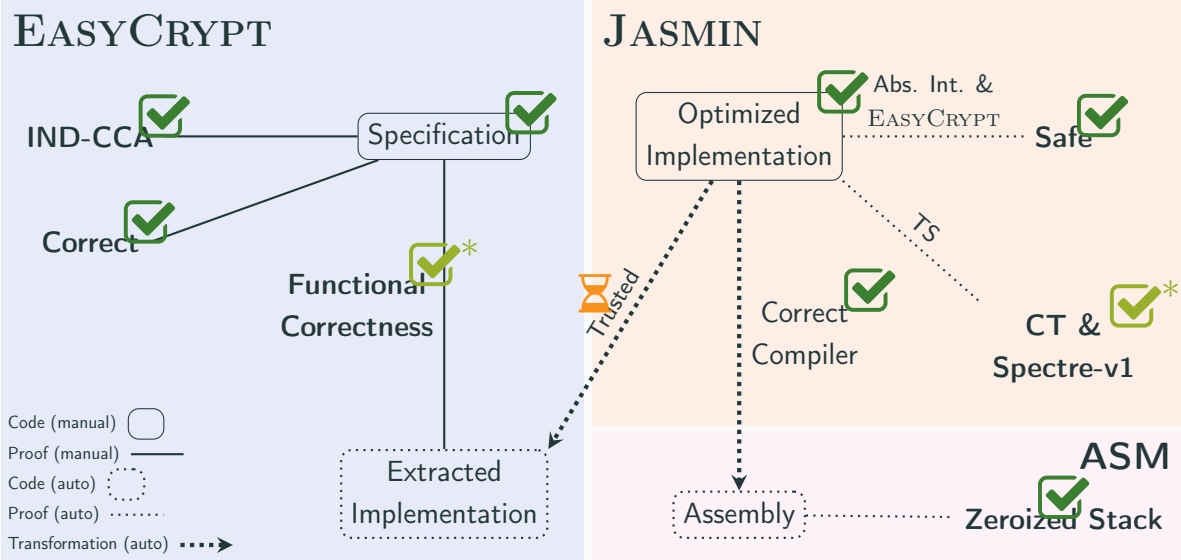
Today's End-to-End Results for ML-KEM



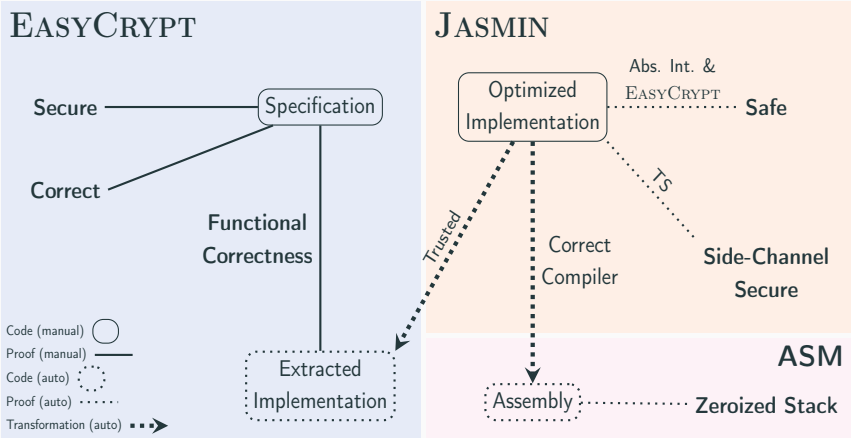
Today's End-to-End Results for ML-KEM



Today's End-to-End Results for ML-KEM

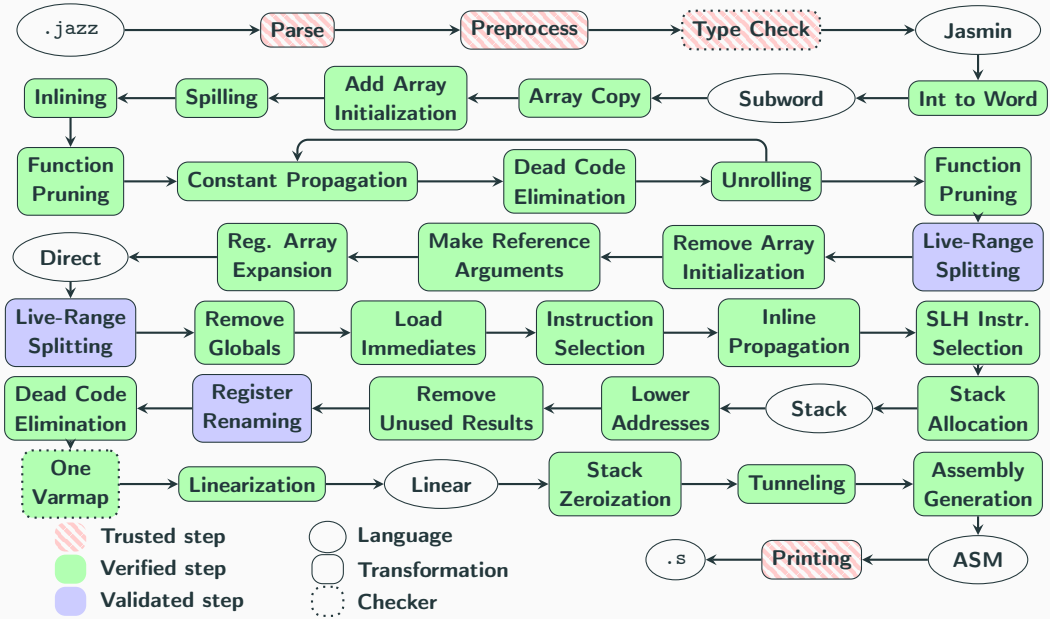


Formosa: End-to-End High-Assurance High-Speed Crypto Software



FORMOSA CRYPTO

formosa-crypto.org


.jazz
Parse
Preprocess
Type Check
Jasmin
Int to Word
Subword
Array Copy
Add Array Initialization
Spilling
Inlining
Function Pruning
Constant Propagation
Dead Code Elimination
Unrolling
Function Pruning
Live-Range Splitting
Remove Array Initialization
Make Reference Arguments
Reg. Array Expansion
Direct
Live-Range Splitting
Remove Globals
Load Immediates
Instruction Selection
Inline Propagation
SLH Instr. Selection
Stack Allocation
Stack
Lower Addresses
Remove Unused Results
Register Renaming
Dead Code Elimination
One Varmap
Linearization
Linear
Stack Zeroization
Tunneling
Assembly Generation
ASM
Printing
.s
Trusted step
Verified step
Validated step
Language
Transformation
Checker

libjade: Formally Verified Assembly

An Open-Source High-Assurance Cryptographic Library (SUPERCOP-like C API)

- Cryptographic hash functions and XOFs: SHA-2, SHA-3, SHAKE
- One-time authenticators and stream ciphers: Poly1305, ChaCha, Salsa
- Authenticated encryption: XSalsa20Poly1305
- Elliptic curves: Curve25519
- Post-quantum KEM and signatures: ML-KEM, ML-DSA, SLH-DSA